Introduction

The **Cangjie Multiplatform** technology enables Cangjie developers to incorporate their code into new Android/iOS applications and/or gradually replace the Java/Objective-C parts of existing applications with Cangjie code. The central mechanism that enables the cross-language, cross-runtime interoperability is *mirror types*, which expose the types of one language to the other.

On the Cangjie side, mirror types enable the inheritance of Java/Objective-C classes with method overriding, as well as the implementation of Java interfaces and Objective-C protocols, all while using conventional Cangjie syntax. On the Java/Objective-C side, the mirror types that represent Cangjie types are expressed in terms of the respective language's own types. All in all, this enables seamless transitions between the Java/Objective-C and Cangjie parts of the application, which includes usage of the target O/S APIs in Cangjie code.

The Challenge and the Solution

Java, Objective-C and Cangjie are object-oriented languages that support inheritance and polymorphism. However, the differences in their semantics, object models, and execution models preclude direct usage of Cangjie objects in Java or Objective-C code and vice versa.

Then, all three languages have managed runtimes that support automatic memory management, threading, exception handling and other low-level features, but again they do that differently. Making two separately designed complex language runtimes aware of each other might push the complexity of the entire system beyond human comprehension.

Instead, the interoperability between Cangjie and Java is achieved by disguising them both as low-level languages. Cangjie and Java parts of the application see each other through the lens of the Java Native Interface (JNI), originally designed to facilitate the development of Java `native` methods in languages such as C and C++. JNI is a rich, but low-level API, and writing `native` Java methods is known to be cumbersome. Fortunately, CJMP automates away the complexity of JNI.

The Objective-C Runtime module API serves a similar role on iOS. It is also designed specifically to enable the creation of bridge layers between Objective-C and other languages. And just like with JNI, CJMP takes care about the cumbersome parts.

Key Concepts

Mirror Types

Considering Cangjie and Java as a pair of interoperating languages, a *mirror type* `T'` defined in one language of the pair is a type that represents an existing type

T defined in the other language, enabling the code written in the first language to use that type, possibly with some limitations.

The Boolean types and numeric types that are essentially the same in both languages naturally mirror each other: the Java mirror type for the Cangjie type `Int32` is `int` and vice versa. Mirroring of numeric types that the other language does not naturally support is not currently possible, so e.g. the Cangjie type `Float16` cannot be mirrored to a Java primitive type.

For a user-defined type such as class, struct or interface, the mirror type would be its closest equivalent in the other language. For instance, the only Java type that can approximate a Cangjie struct or tuple type with reasonable accuracy is a `final` Java class.

A mirror type exposes the members and constructors of the original user-defined type that are accessible *and* can be used in the other language, so again a Cangjie member function that returns a value of type `UInt64` cannot be mirrored to Java without some trickery.

Normally you would obtain mirror type definitions for a Java or Cangjie type *and* its required dependencies automatically. For Java types, a standalone mirror generator is provided, whereas Java mirrors of Cangjie types are produced by the `cjc` compiler as a byproduct of the normal compilation process.

Mirroring Java Types to Cangjie

The `cjc` compiler replaces any uses of Java mirror types with the appropriate glue code, so only the names of types themselves and names and types of their accessible members matter. Therefore, mirrors of Java types only contain member declarations, not definitions: constructors and member functions/properties have no bodies and member variables have no initializers. For the same reason, `private` and package-private members are omitted.

For such an extension of the regular Cangjie syntax to work, each mirror type must be marked with a `@JavaMirror` annotation. It helps the compiler distinguish between mirror type declarations and normal Cangjie type definitions.

For example, a mirror class for the following Java class:

```
public class Node {
    public static final int A = 0xDeadBeef;
    private int _id;
    public Node(int id) { _id = id; }
    public int id() { return _id; }
}
```

might look like this:

```
@JavaMirror
public open class Node {
```

```
    public static let A: Int32
    public init(id: Int32)
    public open func id(): Int32
}
```

A few foundational Java types, namely `java.lang.Object`, `java.lang.String` and arrays, have predefined mirrors in the interoperability library.

Mirroring Cangjie Types to Java Types

Unlike the mirrors of Java types, for which `cjc` has dedicated support, mirrors of Cangjie types must look exactly like normal Java types from the outside, for the Android toolchain to pick them up. The `cjc` compiler therefore generates those mirrors in the form of Java source code files containing glue code responsible for linking together the Java and Cangjie worlds.

For instance, if the class `Node` from the example in the previous section was originally defined in Cangjie as follows:

```
public class Node {
    private let _id: Int
    public prop id: Int {
        get() { _id }
    }
    public init(id: Int) { this._id = id }
}
```

the `cjc` compiler could generate the following Java mirror for it:

```
public final class Node {
    /* Some glue code */
    public Node(long id) {
        /* Glue code constructing a Cangjie Node instance and associating
         * it with the Java Node instance being constructed, i.e. 'this'.
         */
    }
    public long getId() {
        /* Glue code invoking the Cangjie 'id' property getter of the
           associated Cangjie Node instance and returning the result.
         */
     }
    /* More glue code */
}
```

Interop Classes

An *interop class* is essentially a Cangjie class that is derived from one or more mirror types and is usable from Java. All its constructors and non-inherited public member functions are exposed to Java code via a conjugate *wrapper class*,

automatically generated by the `cjc` compiler. The wrapper class itself defines no other user-callable methods or constructors, but any methods it may have inherited from its supertypes may be called from both Java and Cangjie code.

For example, when compiling the following Java interop class:

```
@JavaImpl
public class BooleanNode <: Node {
    private let flag: Bool
    public init(id: Int32, flag: Bool) {
        super.init(id)
        this.flag = flag
    }
    public func getFlag(): Bool {
        flag
    }
}
```

the `cjc` compiler will also yield a Java source code file similar to the following:

```
public class BooleanNode extends Node {
    /* glue code */
    public BooleanNode(int id, boolean flag) {
        /* Glue code constructing a Java BooleanNode instance and associating
         * it with the Cangjie instance being constructed, i.e. 'this'.
         */
    }
    public boolean getFlag() {
        /* Glue code invoking the 'getFlag' member function of the associated
           Cangjie BooleanNode instance and returning the result.
         */
    }
    /* more glue code */
}
```

Foreign Types

The mirror types and interop classes are not perfectly native to the language in which they are defined. Extending the analogy, their status is more like temporary visitors and work visa holders respectively, so they are collectively called *foreign types* throughout this document.

Java-Compatible Types

The following Cangjie types are called *Java-compatible*:

- Value types that have direct equivalents among Java primitive types (`Int16` is included, but `UInt8` is not)

- Foreign types
- Types of the form `Option<T>` where *T* is a foreign type (see ***null*** *Handling* for reasoning)

The special generic mirror type `JArray<T>` included in the interop library represents Java arrays. Its type variable `T` must be a Java-compatible type.

For obvious reasons, the parameters and return values of public member functions of a foreign type may only have types compatible with the respective language.

The member variables of an interop class may have any types. Their public member variables that have Java-compatible types may be accessed from both Java and Cangjie.

```
@JavaImpl
class NamedNode <: Node {
    public var jName: JString  // resides in the conjugate wrapper class instance
    public var cjName: String  // resides in the interop class instance
    init(name: JString) {
        jName = name
        cjName = name.toCangjieString()
    }
}
```

Interop Scenarios

There are essentially two Java-Cangjie interoperation scenarios:

- In one scenario, Cangjie types and functions are exposed to Java, enabling the use of Cangjie APIs, libraries and application components in Java code.
- In the other scenario, Java classes and interfaces are exposed to Cangjie, enabling Cangjie code to access Android APIs, third party Java libraries and components of the application that remain in Java.

The above scenarios are not mutually exclusive, one application may as well be utilizing both. Also, class inheritance and interface implementation are supported in both scenarios (a Java class may subclass a Cangjie class and vice versa). That enables seamless callbacks, so neither scenario is unidirectional. However, each scenario has its own feature set, limitations, tooling, etc., so they are described separately.

Using Cangjie in Java

To enable access to a Cangjie library, API, or some other piece of code from Java, you need to generate Java mirror types for the Cangjie types that you want to expose. The `cjc` compiler can do that for you as described in Cangjie Mirror Generation Reference.

NOTE: The type system of the Cangjie language is richer and more versatile than its Java counterpart. There are also some principal differences in the features shared by the two languages, most notably generics. Expressing certain features of Cangjie in Java is therefore impossible, and other features can be difficult to express in a natural and compact manner. See Cangjie to Java Mapping for the exact list of supported features and associated restrictions.

Example:

Suppose you want to expose to Java a Cangjie struct `Vector` defined as follows:

```
package cj

public struct Vector {
    private var x: Int32 = 0
    private var y: Int32 = 0

    public init(x: Int32, y: Int32) {
        this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector {
        Vector(x + v.x, y + v.y)
    }
}
```

When compiling `Vector.cj`, you would invoke the `cjc` compiler with two additional options: `--experimental --enable-interop-cjmapping=Java`. The compiler would then generate an extra file, `Vector.java`, similar to the following:

```
package cj;

public final class Vector {
    // glue code

    public Vector(int x, int y) { /* glue code */ }

    public Vector add(Vector v) { /* glue code */ }

    // more glue code
}
```

Now you can add `Vector.java` to your Android project under `src/main` and use the class `Vector` in your Java code as if it was a normal Java class: define variables of that type, create instances, pass them around, including to Cangjie code such as the method `Vector.add`, apply type comparison and type conversion operators, etc.

Limiting the Exposure

The option `--enable-interop-cjmapping` instructs the `cjc` compiler to generate mirror types for *all* `public` Cangjie types that it compiles, and expose *all* `public` member functions and constructors of those types. Oftentimes, however, such total exposure is undesirable, as what you want to use in Java may be just a small percentage of the functionality provided by a Cangjie library or framework.

You may precisely control the exposure of Cangjie types and their members by passing the pathname of a dedicated configuration file to `cjc` via the option `--import-interop-cj-package-config-path`. That file is a plain text file the contents of which must conform to the TOML syntax.

In the `[default]` section of that file, you can specify the default API exposure strategy for all Cangjie packages, and then modify it for particular packages in the respective `[[package]]` entries. For example:

```
[default]
APIStrategy="None"        # Expose nothing by default

[[packages]]
name="com.example.pkg1"   # From this specific package,
APIStrategy="Full"        # expose everything
   .  .  .
```

Furthermore, for each package you can use either the `included_apis` or `excluded_apis` property to respectively expose or hide particular types and/or members:

```
   .  .  .
[[packages]]
name="com.example.pkg2"           # From this specific package,
included_apis = [ "Vector",       # Only the type 'Vector'
                  "Vector.add"    # and its member function 'add'
                ]                 # are exposed

[[packages]]
name="com.example.pkg3"           # From this specific package,
APIStrategy="Full"                # everything
excluded_apis = [ "TopSecret",    # but the type 'TopSecret' and
                  "Auth.getPwd"   # member function 'Auth.getPwd'
                ]                 # are exposed
```

Refer to Cangjie Mirror Generation Reference for more details.

Generic Cangjie Types Mirroring

Many important Cangjie types are parameterized. Unfortunately, certain fundamental differences between Cangjie and Java generics preclude mirroring of the former into the latter. As a workaround, the Cangjie SDK supports *monomorphization* of mirror types: it generates a separate *non-generic* Java mirror type for each supplied set of valid type arguments for the given parameterized Cangjie type.

For instance, if you have a generic Cangjie class `p.Pair<T,U>`:

```
package p
public class Pair<T,U> { . . . }
```

and want to manipulate instances of `Pair<Int, String>` and `Pair<Foo, Bar>` in your Java code, you would add a `generic_object_configuration` property to the `[[package]]` entry for `p` in the configuration file, with the following content:

```
#   .  .  .
[[package]]
name = "p"
generic_object_configuration  = [
    { name = "Pair",
      type_arguments = [
          "Int, String",
          "Foo, Bar"
      ] },
#       .  .  .
]
```

`cjc` would then generate two separate Java classes:

```
public final class GIntString { . . . }

public final class GFooBar { . . . }
```

representing those particular *instantiations* of the type `Pair<T,U>`.

NOTE: The current version only supports boolean and numeric types as type arguments of monomorphized generic types, so it cannot actually expose `Pair<Foo, Bar>`.

Refer to Generics Instantiations for more details.

Cangjie to Java Mapping

The current version of the `cjc` compiler uses Cangjie → Java mapping described in this chapter. See Cangjie Mirror Generation Reference for applicable `cjc` command-line options.

General Considerations

The Cangjie type system is much more versatile than that of the Java language. Many Cangjie types and their features do not have direct or even reasonably close equivalents in Java, so some Cangjie types have limited support in the current version and some are not supported at all.

Consequently, if the type of a public member, function/constructor parameter, or function return type is not supported, the respective member, function or constructor cannot be mirrored. The compiler reports such usages as errors. However, if such a type is used in an unsupported *context*, e.g. as the type of a public member variable, the respective entity is simply not mirrored and no error is reported.

Type Aliases

Type alias declarations are *not* mirrored, any type alias usages are treated as if the aliased types were substituted.

Names

The original names of Cangjie packages, functions, types and type members are currently preserved, which means that some of them may clash with Java keywords, such as "`int`" or `final`, or contain characters that are not permitted in Java names.

Boolean and Numeric Types

Cangjie Boolean and numeric types that have equivalents among Java primitive types are mirrored to those respective primitive types; those that don't are not supported, as well as the types `IntNative` and `UIntNative`:

Cangjie Type	Java Type
`Bool`	`boolean`
`Int8`	`byte`
`Int16`	`short`
`Int32`	`int`
`Int64`	`long`
`IntNative`	Not supported
`UInt8`	Not supported
`UInt16`	`char`
`UInt32`	Not supported
`UInt64`	Not supported
`UIntNative`	Not supported
`Float16`	Not supported
`Float32`	`float`

Cangjie Type	Java Type
`Float64`	`double`

See General Considerations for information about the handling of unsupported types.

Rune

The type `Rune` is not supported.

See General Considerations for information about the handling of unsupported types.

String

The type `String` is not supported yet.

See General Considerations for information about the handling of unsupported types.

Array

Array types (`Array<T>` instantiations) are not supported yet.

See General Considerations for information about the handling of unsupported types.

Tuple

Cangjie tuple types are mirrored into `final` Java class definitions. Those definitions are generated automatically by the `cjc` compiler and contain glue code that transfers control and data between Java and Cangjie. They should not be modified manually.

```
package tuples

import interoplib.interop

public class SimpleTupleExample {
    public static func z(): (Int, Int) {
        (0, 0)
    }
}
```

```
// java-gen/SimpleTupleExample.java
package tuples;
// Glue code
```

```java
public class SimpleTupleExample {
    // Glue code
    public static native TupleOfInt64Int64 z();
    // Glue code
}

// java-gen/TupleOfInt64Int64.java
package tuples;
// Glue code
final public class TupleOfInt64Int64 {
    // Glue code
    public TupleOfInt64Int64(long item0, long item1) {
        // Glue code constructing a Cangjie (Int, Int) tuple and associating it
        // with the Java object being constructed, i.e. 'this'.
    }

    public long item0() {
        // Glue code retrieving the 0th element from the Cangjie (Int, Int)
        // tuple associated with 'this' and returning it.
    }

    public long item1() {
        // Glue code retrieving the 1th element from the Cangjie (Int, Int)
        // tuple associated with 'this' and returning it.
    }
    // Glue Code
}
```

A tuple type `(`*`T0`*`, `*`T1`*`,... `*`Tn`*`)` is mirrored into a `final` Java class with the following properties:

- If any *`Ti`* is a type alias, the tuple type is mirrored as if that `Ti_` was the aliased type. For instance, `(Int, Int)` is processed as if it was actually `(Int64, Int64)`.
- The name of the mirror class is `TupleOf` *`T0T1...Tn`* (that is, the names of *Cangjie* types of tuple elements are concatenated and prefixed with `TupleOf`). For instance, the name of the Java mirror class for the Cangjie tuple type `(Int, Int)` will be `TupleOfInt64Int64` due to the aforementioned alias replacement, as in the above example.

  NOTE: Types that do not have their own simple names (*not* aliases) are not supported as tuple element types yet. See General Considerations for information about the handling of unsupported types.
- The mirror class has a single `public` constructor with parameters of types *`T0'`*, *`T1'`*, ... *`Tn'`*, where each *`Ti'`* is the Java mirror type for *`Ti`*. It creates a Cangjie tuple of the mirrored type, with values of its elements

derived from constructor arguments, and associates it with the Java object that is being constructed, i.e. `this`. See the example above.

- For each element type *`Ti`*, the mirror class has a parameterless instance method `public` *`Ti'`* `item`*`i`*`()`, which returns the mirror of the *`i`*-th element of the Cangjie tuple associated with the receiver during its construction. *`Ti'`* is again the Java mirror type for *`Ti`*.

The current implementation has the following limitations:

- Support for tuple element types is limited to numeric and boolean types. See General Considerations for information about the handling of unsupported types.
- The compiler does not mirror tuple types automatically as it encounters their usages in the definitions of other mirrored entities. All such tuple types must be listed explicitly in the configuration file:

  ```
  #   .   .   .
  [[package]]
  name = "tuples"
  tuple_configuration = [ "(Int, Int)" ]
  #   .   .   .
  ```

  The compiler reports an error upon encountering a tuple type that must be mirrored but is not listed in the `tuple_configuration` array for the respective Cangjie package.
- Cangjie tuple types are *structural*, that is, two tuple types are compatible if they have the same number of elements and their respective element types are the same. This is also true for aliases of tuple types. However, Java class types that mirror Cangjie tuples are not in a subclass relationship and are therefore incompatible. This may lead to a problem if essentially the same tuple type is cross-used among two or more Cangjie packages exposed to Java.
- Cangjie tuple types are value types, whereas their mirrors are Java reference types. The consequences are two-fold:
  - Multiple Java variables may refer to a single Cangjie tuple. That itself should pose no problems because tuples are immutable.
  - The comparison operators `==` and `!=` have totally different semantics. In Java, they test mirror class instances for referential equality, whereas in Cangjie tuples are compared element by element. In fact, two structurally identical Cangjie tuples may be uncomparable if some of their element types do not support the respective operator.

    Furthermore, in the current implementation the mirror classes for tuple types do not implement the method `equals` (more precisely, the generated overriding method throws an `UnsupportedOperationException`),

so there is no concise way to compare two tuples of the same type on the Java side.

Range

Range types are not supported yet.

See General Considerations for information about the handling of unsupported types.

Special Types

The type `Unit` is only supported as a function return type, mapped to Java `void`.

The type `Nothing` cannot be supported.

The type `Any` is not supported yet.

See General Considerations for information about the handling of unsupported types.

Function Types

Cangjie function types are mirrored into Java functional interface declarations:

```
package cjworld

import interoplib.interop

public class IntFuncBox {
    private let _f: (Int) -> Int
    public init(f: (Int) -> Int) { _f = f }
    public func unbox(): (Int) -> Int { _f }
}
```

```
package cjworld;
// Glue code

@FunctionalInterface
public interface Int64ToInt64
{
// Glue code

    public long call(long p1);

// Glue code
}
```

```
package cjworld;
// Glue code

public class IntFuncBox {
// Glue code

    public IntFuncBox(Int64ToInt64 f) {
        // Glue code creating an instance of the Cangjie class IntFuncBox
        // and associating it with the object being constructed, i.e. 'this'.
    }

    public Int64ToInt64 unbox() {
        // Glue code calling the unbox() instance member function of the
        // Cangjie object associated with 'this' and wrapping it into an
        // instance of a helper class that implements Int64ToInt64.
    }

//  Glue code
}
```

A function type `(`*`T1`*`, `*`T2`*`,... `*`Tn`*`) -> U` is mirrored into a Java functional interface with the following properties:

- If any *`Ti`* is a type alias, the function type is mirrored as if that *`Ti`* was the aliased type. For instance, `(Int) -> Int` is processed as if it was actually `(Int64) -> Int64`.
- If any *`Ti`* and/or *`U`* is a type variable, the function type is mirrored separately for each monomorphization of the respective generic type or member. See the example in Generics Instantiations.
- The name of the mirror interface is *`T1T2...Tn`*`To`*`U`* (that is, the names of *Cangjie* types of parameters are concatenated, followed by `To` and then followed by the name of the result type). For instance, the name of the Java mirror interface for the Cangjie function type `(Int, Bool) -> Int` will be `Int64BoolToInt64` due to the aforementioned alias replacement.

  NOTE: Types that do not have their own simple names (*not* aliases) are not supported as function element types yet. See General Considerations for information about the handling of unsupported types.
- The mirror interface has a single `public` instance method:

  U' `call(`*`T1' p1, T2' p2, ... Tn' pn`*`) { ... }`

  where each *`Ti'`* is the Java mirror type for *`Ti`* and *`U'`* is the Java mirror type for *`U`*, or `void` if *`U`* is `Unit`. The parameters are named automatically: `p1`, `p2`, ... `pn`.

- The mirror interface type can be used to pass functions and lambda expressions both ways:
  - A value of that type transferred from Cangjie to Java, e.g. returned from a mirrored Cangjie member function called from Java, is associated with a value of a *Cangjie* function type, e.g. a lambda. The method `call()` invokes that Cangjie function, passing its arguments over as values of the mirrored Cangjie types *`T1`*`, `*`T2`*`,... `*`Tn`*, and returns a mirror of the value of type *`U`* returned from that Cangjie function (or nothing if that function returns `Unit`).
  - The mirror type is a regular Java functional interface type in all aspects, so not only a Java class may implement it, but lambda expressions of a matching type are also compatible with it. When an instance of such a Java class or a compatible lambda expression is passed over to Cangjie, a value of the mirrored Cangjie function type, `(`*`T1`*`, `*`T2`*`,... `*`Tn`*`) -> U`, gets synthesized and associated with that Java lambda or class instance, and Cangjie code calling it ends up calling the Java lambda or method.

The current implementation has the following limitations:

- Support for function parameter and return value types is limited to supported numeric types, `Bool`, and `Unit`. See General Considerations for information about the handling of unsupported types.
- The compiler does *not* mirror function types automatically as it encounters their usages in the definitions of other mirrored entities. All such types must be listed explicitly in the configuration file:

  ```
  #   .   .   .
  [[package]]
  name = "lambdas"
  lambda_patterns = [
      { signature = "(Int) -> Int" },
      { signature = "(Float64) -> Bool" },
  #   .   .   .
  ]
  ```

  The compiler reports an error upon encountering a function type that must be mirrored, but is not listed in the `lambda_patterns` array for the respective Cangjie package.
- Cangjie function types are *structural*, that is, two function types are compatible if they have the same number of parameters, their respective parameter types are the same, and their return types are the same. This is also true for aliases of function types. However, two Java functional interface types that mirror essentially the same Cangjie function type used in different packages are incompatible. This may lead to a problem if such

a function type is cross-used among two or more Cangjie packages exposed to Java.

- Cangjie function types are uncomparable, whereas their Java mirrors are. That should pose no problems.

Struct Types

Public Cangjie struct type definitions are mirrored into `final` Java class definitions. Those definitions are generated automatically by the `cjc` compiler and contain glue code that transfers control and data between Java and Cangjie. They should not be modified manually.

```
package cj

import interoplib.interop.*

public struct Vector {
    private var x: Int32 = 0
    private var y: Int32 = 0

    public init(x: Int32, y: Int32) {
        this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector {
       let res = Vector(x + v.x, y + v.y)
       print("cj: (${x}, ${y}) + (${v.x}, ${v.y}) = (${res.x}, ${res.y})\n", flush: true)
       return res
    }

    public static func dump(v: Vector): Unit {
        print("cj: Hello from static func in cj.Vector (${v.x}, ${v.y})\n", flush: true)
    }
}
```

```
package cj;

// Glue code imports

final public class Vector {
    // Glue code

    public Vector(int x, int y) {
        // Glue code that constructs an instance of the Cangjie Vector struct
        // and associates it with the Java object 'this'
```

```
    }

    public Vector add(Vector v) {
        // Glue code that retrieves the associated Cangjie Vector struct
        // instances associated with `this` and `v`, calls the add() instance
        // member function of the Cangjie-this, passing the Cangjie-v to it
        // as a parameter, and, finally, creates a new Java Vector instance
        // and associates it with the result of the add() call.
    }

    public static void dump(Vector v) {
        // Glue code that calls the dump() static member function of the
        // Cangjie struct Vector, passing to it the instance associated
        // with 'v'/
    }

    // More glue code
}
```

Only public struct members and common constructors are mirrored.

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. The modifiers `public` and `static` are preserved.

Common constructors are mirrored into constructors with the respective mirror types substituted for parameter types. *This includes the default constructor that might have been implicitly declared.* The modifier `public` is preserved.

The current version imposes a number of severe limitations on the struct types that can be mirrored, to the extent that it may be fair to say that a struct type needs to be designed specifically for exposing it to Java:

- Member variables are not mirrored and no means for accessing them is provided. This limitation will be removed in a future version. In the meantime, you may add getter functions as a workaround.
- Mirroring of `mut` instance member functions is a work in progress. They are mirrored, but incorrect glue code is generated in some circumstances.
- Mirroring of structs that implement interfaces other than `Any` is not supported. An attempt to mirror a struct that explicitly implements an interface results in a compile-time error.
- Mirroring of generic structs is supported through monomorphization. See Generics for details.
- Mirroring of structs that contain public member properties, member operator functions, or primary constructors is not supported. An attempt

to mirror a struct containing an entity of one or those kinds results in a compile-time error.

- An attempt to mirror a struct member function or constructor with an unsupported parameter type, or a member function with an unsupported return type, results in a compile-time error.

Classes and Interfaces

Cangjie class and interface definitions are mirrored respectively into Java class and interface definitions. Those definitions contain automatically generated glue code and should not be altered in any way.

```
package cj

import interoplib.interop.*

public interface Valuable {
    public func value(): Int
}

public open class Singleton <: Valuable {
    private let _v: Int
    public init (v: Int) { _v = v }
    public func value(): Int { _v }
}

public class Zero <: Singleton {
    public init() { super(0) }
}
```

```
// Valuable.java
package cj;

// Glue code imports

public interface Valuable {
    public long value();
}

// Glue code
```

```
// Singleton.java
package cj;

// Glue code imports

public class Singleton implements Valuable {
```

```
    // Glue code

    public Singleton(long v) {
        // Glue code that constructs an instance of the Cangjie Singleton class
        // and associates it with the Java object 'this'
    }

    public long value() {
        // Glue code that retrieves the associated Cangjie Singleton class
        // instance, calls its member function value() and returns the result
    }

    // Glue code
}
```

```
package cj;

// Glue code imports

public final class Zero extends Singleton {
    // Glue code

    public Zero() {
        // Glue code that constructs an instance of the Cangjie Zero class
        // and associates it with the Java object 'this'
    }

    // Glue code
}
```

Identifiers that serve as names of mirrored types and their members are currently preserved even if they clash with Java keywords such as `byte`.

From the outside, the mirror types are normal Java classes and interfaces in all aspects: except for `final` classes, they can be extended/implemented, their methods can be overloaded with conventional Java methods in subtypes, their non-`final` methods can be overridden, types of their instances can be compared with `instanceof`, and so on.

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. Methods that mirror non-`open` member functions are modified with `final`. The modifiers `public` and `protected` (see below) are preserved. The modifier `static` is preserved for member functions of classes. Parameter names are preserved. **WARNING:** `static` interface member functions are currently mirrored into instance methods, but incorrect code is generated for mirrors of Cangjie classes that implement such interfaces.

Member properties themselves are not mirrored into anything, but their getters and setters are mirrored into methods. The names of those methods start respectively with "`get`" or "`set`", followed by the property name with the first letter capitalized to maintain camel case. A method mirroring a property getter is parameterless and returns a value of the type mirroring the property type, whereas a property setter mirror method is `void` and takes a single parameter of that same mirror type. The modifiers `public`, `protected` (see below) and `static` are preserved.

Constructors are mirrored into constructors with the respective mirror types substituted for parameter types. *This includes the default constructor that might have been implicitly declared.* The modifier `public` is preserved. Parameter names are preserved.

Generally, only the following entities are subject to mirroring:

- `public` constructors and members of the above listed kinds
- `protected` `open` members of `open` classes (this includes `protected` abstract member functions), as they can be overridden on the Java side.

Member variables and **Member operator functions** are currently not mirrored.

As Cangjie does not support type nesting, unlike Java, mirror types never have nested types (except maybe for `private` member types used by glue code).

The current version imposes a number of severe limitations on the class and interface types that can be mirrored, to the extent that it may be fair to say that such types need to be designed specifically for exposing them to Cangjie:

- `abstract` classes are not mirrored.
- Member variables are not mirrored and no means for accessing them is provided. This limitation will be removed in a future version. In the meantime, you may add getter/setter functions to Cangjie classes manually as a workaround.
- Mirroring of generic classes is supported through monomorphization. See Generics for details.
- Mirroring of member operator functions is not supported. An attempt to mirror a class or interface containing such an entity results in a compile-time error.
- An attempt to mirror a class member function or constructor with an unsupported parameter type, or a member function with an unsupported return type, results in a compile-time error.

Enums

A Cangjie enum is mirrored into a `final` Java class without a `public` constructor.

```
public enum TimeUnit {
    | Year(Int64)
    | Month(Int64)
    | Year
    | Month
}
```

```
public final class TimeUnit {
    /* Glue code */

    public static TimeUnit Year(long p1) {
         // Glue code creating a Cangjie Year(p1) enum and associating it
         // with a newly created Java TimeUnit instance
    }

    public static TimeUnit Month(long p1) {
         // Glue code creating a Cangjie Month(p1) enum and associating it
         // with a newly created Java TimeUnit instance
    }

    public static TimeUnit Year =
        // Glue code creating a Cangjie Year enum and associating it
        // with a newly created Java TimeUnit instance

    public static TimeUnit Month =
        // Glue code creating a Cangjie Year enum and associating it
        // with a newly created Java TimeUnit instance

    /* More glue code */
}
```

Enum constructors without parameters are mirrored into `static` variables initialized with instances of the class associated with the respective constructors of the Cangjie enum.

Enum constructors with parameters are mirrored into `static` factory methods with the same names and matching types and numbers of parameters; names of parameters are synthesized (`p1`, `p2`, ...).

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. The modifiers `public` and `static` are preserved.

Member properties themselves are not mirrored into anything, but their getters (enums cannot have `mut` properties) are mirrored into methods. The names of those methods start with "`get`", followed by the property name with the first letter capitalized to maintain camel case. A method mirroring a property

getter is parameterless and returns a value of the type mirroring the property type. The modifiers `public` and `static` are preserved.

The current version imposes a number of limitations on the enum that can be mirrored:

- Mirroring of enums that implement interfaces other than `Any` is not supported. An attempt to mirror an enum that explicitly implements an interface results in a compile-time error.
- Mirroring of generic enums is supported through monomorphization. See Generics for details.
- Mirroring of enums that contain member operator functions is not supported. An attempt to mirror an enum containing such an entity results in a compile-time error.
- An attempt to mirror an enum member function or constructor with an unsupported parameter type, or a member function with an unsupported return type, results in a compile-time error.

Members defined in enum extensions (`extend`) are currently mirrored as if they were members of the enum itself.

Recursively defined enums are supported:

```
public enum Peano {
    | Z
    | S(Peano)

    public func toInt(): Int {
        match (this) {
            case Z => 0
            case S(x) => 1 + x.toInt()
        }
    }
}
```

Generics

Java and Cangjie generics are fundamentally different, so the latter cannot be mirrored into the former. However, *specific instantiations* of parameterized Cangjie types, such as `G<Int64>`, are concrete types, and therefore can be mirrored into non-generic Java types. Such types are also called *monomorphized generics*.

Use the respective configuration file of the `cjc` compiler to specify which instantiations of which types to mirror. Refer to Generics Instantiations for details.

Example:

When compiling the following Cangjie class definition:

```
public class G<T> {
    private let t: T
    public init(t: T) { this.t = t }
    public func get(): T { t }
}
```

with the following lines in the respective section of the configuration file:

```
    . . .
    generic_object_configuration  = [
        { name = "G", type_arguments = ["Bool", "Int"] },
    ]
    . . .
```

the compiler will generate Java mirror classes for `G<Bool>` and `G<Int>`, named respectively `GBool` and `GInt`.

Refer to the Cangjie Mirror Generation Reference section for complete instructions.

`null` Handling

As Cangjie has no null type, passing `null` as a parameter to a mirrored Cangjie method results in a run-time exception in glue code:

```
public class Node {
    private let next: ?Node
    public init() {
        next = None
    }
    public init(next: Node) {      /* Pure Cangjie */
        this.next = Some(next)
    }
}
```

```
public final class Node {
    Node() { /* glue code */ }
    Node(Node next) { /* glue code */ }
}
```

```
    . . .
     Node list = new Node(null);   /* NullPointerException */
```

Conversely, there is currently no way to return `null` from a `public` member function of a Java-compatible type.

NOTICE: The current version supports automatic `Option<T>` (un)wrapping with `null` mapped to `None`, but only for Java reference types mirrored to Cangjie, not the other way round. See `null` Handling for details.

Cangjie Mirror Generation Reference

There is no standalone mirror generator facilitating the exposure of Cangjie types to Java and Objective-C code in the Cangjie SDK. The `cjc` compiler itself is capable of generating Java/Objective-C mirror type definitions for Cangjie types right when it compiles the Cangjie source code of those types.

Command-line Syntax

The following additional `cjc` options enable and control the generation of mirrors for Cangjie types during compilation:

`--experimental` *(mandatory)*

This option must be specified as of the current version, since the whole interop feature is still in development, but will not be necessary in the future.

`--enable-interop-cjmapping=Java` *or*
`--enable-interop-cjmapping=ObjC` *(mandatory)*

Enables the generation of mirror type definitions for Cangjie types respectively in the form of Java or Objective-C source code files.

`--output-interop-cjmapping-dir` ***`pathname`*** *(optional)*

The ***`pathname`*** must point to a directory into which the `cjc` compiler shall place its output Java or Objective-C source code files containing mirror type definitions. If ***`pathname`*** does not point to an existing file system location, `cjc` attempts to create a directory there. If ***`pathname`*** points to something other than a directory, `cjc` terminates with an error message.

The default value of ***`pathname`*** is either `./java-gen` or `./objc-gen`, depending on whether `--enable-interop-cjmapping` is set respectively to "`Java`" or "`ObjC`".

`--import-interop-cj-package-config-path` ***`pathname`*** *(optional)*

The ***`pathname`*** must point to a configuration file that defines which Cangjie types are exposed to Java or Objective-C as mirror types. See Cangjie Mirror Generation Configuration for details.

Cangjie Mirror Generation Configuration

A Cangjie mirror generation configuration file is a plain text file in the TOML syntax. It specifies:

- Non-generic Cangjie types to mirror
- Specific instances of generic Cangjie types to mirror

Defaults

The `[default]` table defines the default settings for packages for which either no package-specific settings are provided at all, or those settings don't override the defaults.

Properties:

`APIStrategy` *(optional)*

A string defining whether all public entities defined in a package shall be mirrored by default or not. Valid values:

- `"Full"` *(default)* - all public entities shall be mirrored by default
- `"None"` - no public entities shall be mirrored by default

`GenericTypeStrategy` *(optional)*

A string defining whether generic public entities shall be mirrored by default or not. Valid values:

- `"Partial"` - instantiations explicitly listed in `generic_object_configuration` shall be mirrored
- `"None"` *(default)*- no generic public entities shall be mirrored by default

Packages

Each entry in the `[[packages]]` array specifies the name of a separate source Cangjie package, and, optionally:

- A filter that defines which types from that package to include in, or exclude from, the set of mirrors
- API mirroring strategy (if other than the default)
- Generic type mirroring strategy (if other than the default)
- Specific generic type instantiations to mirror

Properties:

`name` *(mandatory)*

A string containing the name of the *source* Cangjie package to which the settings in this `[[packages]]` array entry apply.

Example:

```
name = "com.example.VectorMath"
```

`APIStrategy` *(optional)*

A string defining whether all public entities of the given package shall be mirrored by default or not. Valid values:

- `"Full"` - all public entities shall be mirrored by default
- `"None"` - no public entities shall be mirrored by default

If this property is absent, the default value is used.

`included_apis` *(optional)*

An array of strings each containing the name of a public entity that needs to be mirrored. If the array contains the qualified name of a type member, the type itself is also mirrored. Either `included_apis` or `excluded_apis` may be present in a given `[[packages]]` array entry, but not both.

Example:

```
included_apis = [
   "Vector.product", // Vector also exposed
   "HiddenV"
]
```

`excluded_apis` *(optional)*

An array of strings each containing the name of a public Cangjie entity defined in the given package that has to be *not* mirrored. Either `included_apis` or `excluded_apis` may be present in a given `[[packages]]` array entry, but not both.

Example:

```
excluded_apis = [
   "Vector.product", // Even if Vector is exposed, product is not
   "HiddenV"
]
```

`GenericTypeStrategy` *(optional)*

A string defining whether generic public entities defined in this package shall be mirrored by default or not. Valid values:

- `"Partial"` - instantiations explicitly listed in `generic_object_configuration` shall be mirrored
- `"None"` - no generic public entities shall be mirrored by default

If this property is absent, the default value is used.

`lambda-patterns` *(optional)*

A (possibly empty) array of tables, each describing a function type that is used by one or more of the exposed entities and therefore must be mirrored.

Each entry of the array has the following mandatory property:

`signature` *(mandatory)*

A string containing a Cangjie function type, such as "(Int) -> Int".

`types`

An array of strings, each containing valid type arguments for the generic type the name of which is specified in the `name` property.

Example:

```
tuple_configuration = [ { signature = "(Int) - > Int" },
                        { signature = "(Float64, Float64) -> Float64" }
                        ]
```

See Function Types for details.

`tuple_configuration` *(optional)*

A list of strings, each containing a tuple type that is used by one or more of the exposed entities and therefore must be mirrored.

Example:

```
tuple_configuration = [ "(Int, Bool)",
                        "(Float64, Float64, Float64)"]
```

See Tuple for details.

`generic_object_configuration` *(optional)*

A list of tables defining the specific instantiations of generic entities to be mirrored. See Generics Instantiations for details.

Generics Instantiations The `generic_object_configuration` property of a `[[packages]]` array entry defines which specific instantiations of generic Cangjie entities will be mirrored.

The value of the property is a (possibly empty) array of tables. Each entry of the array has the following two mandatory properties:

`name`

A string containing the name of a public generic Cangjie type defined in the current package.

`type_arguments`

An array of strings, each containing valid type arguments for the generic type the name of which is specified in the `name` property.

NOTE: The current version only supports numeric types, `Bool`, and `Unit` as type arguments for monomorphization. Specifying any other type in a `types` array element results in a compile time error.

> Simply put, "valid" means that the type/function can be instantiated with the given string between the angle brackets `< >`.
>
> Consider the following Cangjie class:

```
public class G<T> {
    public func f(t: T): Unit {}
    public func f(b: Bool): Unit {}
}
```

`G<Bool>` cannot be instantiated, so

```
generic_object_configuration  = [
    { name = "G", types = ["Bool"] }
]
```

would trigger a compiler error.

Examples:

1. Given the following Cangjie definitions

   ```
   public class G<T> { . . . }
   public func f<T>(): Unit { . . . }
   public struct S<T,U> { . . . }
   ```

   the property

   ```
   generic_object_configuration  = [
       { name = "G", type_arguments = ["Int"] },
       { name = "f", type_arguments = ["Int", "Bool"] },
       { name = "S", type_arguments = ["Int, Bool"] }
   ]
   ```

   instructs the `cjc` compiler to generate mirrors for the following instantiations:

   ```
   G<Int>
   f<Int>
   f<Bool>
   S<Int, Bool>
   ```

   and name them like `GInt`, `fInt`, and so on.

2. Given this Cangjie source code file:

   ```
   package cjworld

   import interoplib.interop.*

   public class Composer<T,U,V> {
       public static func compose(f: (T) -> U, g: (U) -> V): (T) -> V {
           { t: T => g(f(t)) }
       }
   }
   ```

   and the following configuration file:

```
[[package]]
name = "cjworld"
APIStrategy = "Full"
GenericTypeStrategy = "Partial"

lambda_patterns = [
  { signature = "(Int) -> Float64" },
  { signature = "(Float64) -> Bool" }
]

generic_object_configuration = [
    { name = "Composer", type_arguments = ["Int64, Float64, Bool"] },
    { name = "Composer<Int64, Float64, Bool>", symbols = ["compose"] },
]
```

the compiler will generate the following non-parameterized mirror types:

- Functional interface `Int64ToFloat64`:

```
// . . .
@FunctionalInterface
public interface Int64ToFloat64
{
    public double call(long p1);
    // . . .
}
```

- Functional interface `Float64ToBool`

```
// . . .
@FunctionalInterface
public interface Float64ToBool
{
    public boolean call(double p1);
    // . . .
}
```

- Class `ComposerInt64Float64Bool`:

```
// . . .
public class GenClass3Int64Float64Bool {
    // . . .
    public static Int64ToBool compose(Int64ToFloat64 f, Float64ToBool g) {
        // . . .
    }
    // . . .
}
```

Using Java in Cangjie

In the first interoperation scenario, described in Using Cangjie in Java, Cangjie types are mapped to Java types that behave just like any conventional Java classes and interfaces: non-`final` classes can be extended, interfaces implemented, instances created and passed around freely, and so on.

In contrast, due to certain technical limitations imposed by the Android Runtime (JVM), any uses of Java types in Cangjie must be fully confined to the code of constructors and member functions of interop classes. Also, mirror type declarations for Java classes and interfaces are generated by a standalone tool, not the `cjc` compiler.

The process of enabling this second scenario is therefore more involved. Here are the steps you need to take:

1. Design the interoperability layer in terms of *Java* classes and methods.

   developer → `interop layer design` (Java *pseudo-code*)

2. Generate mirror declarations for any existing Java classes and interfaces used in the design of the interoperability layer.

   `.class` → mirror generator → `.cj` (mirrors)

3. Write the classes constituting the interoperability layer *in Cangjie*. Use the mirrored Java types as needed: create instances, call methods, etc.

   `interop layer design` + `.cj` (mirrors) → developer → `.cj` (interop layer)

4. Compile the interop classes and mirror type declarations together with `cjc`. `cjc` will generate:

   - the necessary glue code for all uses of the mirror types.
   - the actual Java source for the Java part of the interoperability layer.

   `.cj` (mirrors + interop layer) → `cjc` → `.so` + `.java` (interop layer)

5. Add to your Android project:

   - `.java` files generated by `cjc`
   - `.so` file generated by `cjc`
   - runtime libraries (`.so` and `.jar`) from the Cangjie SDK

   then insert the necessary uses of the interop layer in the Java source and rebuild the project.

   Android project + `.java`, `.jar`, `.so` → Android toolchain → `.apk`

The sections Initial Interop Class Creation Workflow and Calling Java From Cangjie describe the above process in detail with an end-to-end example.

Initial Interop Class Creation Workflow

Step 1: Design the interoperability layer

On this step, you design the API of one or more interop classes *from the Java code perspective*, i.e. you need to decide, for each interop class:

- To which Java package it will belong;
- What Java class it will extend, if not the default `java.lang.Object`;
- Which Java interface(s) it will implement, if any; and
- What `public`/`protected` methods and/or constructors it will have. You only need to know the types of parameters and, for methods, their names and return value types; the implementations you will write in Cangjie.

See also Features and Limitations of Interop Classes.

Example:

Suppose we want to pass control from Java to Cangjie by invoking a certain `static` method `m()` accepting parameters of types `com.example.a.A`, `java.lang.String`, and `int`, and returning a value of type `com.example.b.B`. We want that method to belong to a class called `Interop` that extends `Object` and belongs to the package `cjworld`.

In other words, we have determined that the Java source code file that we want to be generated on step 4 should have the following content:

```
package cjworld

import com.example.a.A;
import com.example.b.B;

public class Interop {
    /* glue code */
    public static B m(A a, String s, int i) {
        /* glue code invoking the Cangjie implementation of m() */
    }
    /* more glue code */
}
```

Step 2: Generate mirror type declarations

Now, you need mirror type declarations for all Java reference types on which your interop classes depend: their supertypes, types of their method and constructor parameters, etc., and, possibly, the dependencies of those types.

NOTE: Mirror classes for `java.lang.Object` and `java.lang.String` are provided in the interop library, as well as a generic mirror class representing Java arrays. Skip this step if the methods and constructors of your interop classes only

pass to/receive from Cangjie values of primitive Java types, `java.lang.Object`, `java.lang.String`, and/or arrays of the foregoing.

Use the Java mirror generator included in the Cangjie SDK to produce those mirror type declarations as follows:

Command line:

```
java-mirror-gen \
    --package-name <package-name> \
    --class-path <full-application-classpath> \
    --destination <output-directory> \
    <names-of-mirrored-types>
```

or

```
/path/to/jdk/21/bin/java \
java-mirror-gen \
    --package-name <package-name> \
    --class-path <full-application-classpath> \
    --destination <output-directory> \
    -jar <jar-file>
```

where

- `<package-name>` is the desired name of the Cangjie package for all mirrored types.
- `<full-application-classpath>` is the complete classpath of the application that is used during its build, including `android.jar`.
- `<output directory>` is the pathname of the directory where the Cangjie source files containing mirror type declarations will be placed, such as `./src/cj`.
- `<names-of-mirrored-types>` are the *fully qualified names* of the Java reference types on which your interop class design depends, other than `java.lang.Object`, `java.lang.String` or arrays.
- `<jar-file>` is the pathname of a single jar file. All `public` classes and interfaces, the `.class` files of which it contains, are mirrored, along with any dependencies found along the `<full-application-classpath>`.

Example:

Continuing the above example, we notice that our class `cjworld.Interop` depends on:

- `java.lang.Object` as a superclass;
- `com.example.a.A`, `java.lang.String`, and `int` as parameter types; and
- `com.example.b.B` as a return value.

We do not need to mirror primitive types such as `int`, and the classes `java.lang.Object` and `java.lang.String` are pre-mirrored. This leaves us with just the two `com.example` classes to mirror. Suppose we want them to appear in the package `javaworld`.

Mirror generator command line with a fictional application classpath:

```
java-mirror-gen \
    --package-name javaworld \
    --class-path /home/user/Android/Sdk/platforms/android-35/android.jar:App.jar \
    --destination ./src/cj \
    com.example.a.A com.example.b.B
```

This will generate files `src/cj/javaworld/src/A.cj` and `src/cj/javaworld/src/B.cj`, *plus* mirror type declarations for any dependencies those two Java types may have.

Step 3: Write Interop Classes

For each Java class skeleton you designed on step 1, write a new Cangjie class as follows:

1. Use the appropriate package and class names (the Java wrapper class will have the same fully qualified name).
2. Import `java.lang.*`.
3. Import the necessary mirror types, if you generated any on Step 2. Do not import the dependencies for now.
4. Annotate the class with `@JavaImpl`.
5. Make the class inherit the mirror of the desired Java class (by default, it will inherit the built-in mirror of `java.lang.Object`).
6. Use `JObject`, `JString` and `JArray<T>` in place of `java.lang.Object`, `java.lang.String` and Java array types respectively.

Type mapping (`T'` is either the matching primitive type or the respective mirror type):

Java Type (`T`)	Cangjie Type (`T'`)
`boolean`	`Bool`
`byte`	`Int8`
`short`	`Int16`
`char`	`UInt16`
`int`	`Int32`
`long`	`Int64`
`float`	`Float32`
`double`	`Float64`
`Object`	`JObject` or `?JObject`
`String`	`JString` or `?JString`
`class C`	`C'` or `?C'`

Java Type (T)	Cangjie Type (T')
interface I	I' or ?I'
T[]	JArray<T'> or ?JArray<T'>

Use ?<T'> (Option<T'>) types for parameters, return values, and local variables of mirror and interop types that may receive/hold Java null.

Use Unit as a return type to represent a void Java method.

See also Features and Limitations of Interop Classes.

Example:

To continue the above example, the interop class for the function m() would look like this:

```
package cjworld

import java.lang.*
import javaworld.*

@JavaImpl
public class Interop {
    public static func m(a: ?A, s: ?JString, i: Int32): ?B {
        /* Just a dummy implementation for now */
        B()    // Assuming that B has an accessible parameterless constructor
    }
}
```

If you don't need m() to ever return null to its Java caller, remove ? from its return value type.

Step 4: Compile the Interop Class

Command line:

```
cjc --output-type=dylib \
    -p <source-directory> \
    -ljava.lang -linteroplib.interop \
    --output-javagen-dir=<java-output-directory>
```

where

<source-directory> is the pathname of the directory containing the source code of interop classes and mirror type declarations.

<java-output-directory> is the pathname of the directory to which the generated Java source files are placed.

The output is a `.so` file with compiled Cangjie code for the interop classes and Java source files (`.java`) for their Java wrappers.

Example:

```
cjc --output-type=dylib \
    -p src/cj \
    -ljava.lang -linteroplib.interop \
    --output-javagen-dir=src/java
```

The compiler will produce two files: `libcjworld.so` and `src/java/cjworld/Interop.java`.

Step 5: Put it all together

1. Add the following files to your Android project:
   - The Java source files generated on the previous step - to the source tree under `src/main` in accordance with the package names.
   - The `.so` file generated on the previous step - to `src/main/jniLibs/arm64-v8a` (create those subdirectories if they don't exist).
   - All `.so` files from the appropriate subdirectory of `$CANGJIE_HOME/runtime/lib/`, such as `$CANGJIE_HOME/runtime/lib/linux_android31_aarch64_cjnative/` - also to `src/main/jniLibs/arm64-v8a`.
   - The file `libc++_shared.so` from the Android NDK - also to `src/main/jniLibs/arm64-v8a`. (That file is located in `toolchains/llvm/prebuilt/`*`host`*`/sysroot/usr/lib/aarch64-linux-android/` under the Android NDK root, where *`host`* is a string corresponding to the O/S and hardware of the system on which you conduct the build, such as "`linux-x86_64`".
   - The file `$CANGJIE_HOME/lib/library-loader.jar` as a dependency.
2. **IMPORTANT: Enforce the legacy convention of compressing all `.so` files in the APK, or the application will crash trying to load Cangjie libraries.**

   In the Gradle build script of your Android application, usually called `build.gradle.kts`, locate the `android {}` block and insert the following option setting into it, if it is not already present:

```
    . . .
android {
        . . .
    packaging {
        jniLibs {
            useLegacyPackaging = true
        }
    }
```

```
}
    .  .  .
```

3. Re-build the project to ensure everything is correct so far. You may also try deploying it.
4. Add the Java code working with interop classes and re-build your Android project again.

Example:

Now we can add an invocation of `Interop.m()` to our Java code, for control to be passed to its Cangjie implementation in our interop class:

```
    .  .  .
B b = Interop.m(new A(), "Test", 0);
    .  .  .
```

Calling Java from Cangjie

Once you have designed, built and integrated the interoperability layer as described in the previous section, you can add code that uses Java types to the member functions of your interop classes. The type mapping is the same:

Cangjie Type (`T'`)	Java Type (`T`)	Remark
`Bool`	`boolean`	
`Int8`	`byte`	
`Int16`	`short`	
`UInt16`	`char`	
`Int32`	`int`	
`Int64`	`long`	
`Float32`	`float`	
`Float64`	`double`	
`JObject` or `?JObject`	`Object`	
`JString` or `?JString`	`String`	
`T'` or `?T'`	`T`	(*)
`JArray<T'>` or `?JArray<T'>`	`T[]`	(**)

(*) `T'` must be either a mirror type for the Java type `T` or an interop class, for which the source code of its Java wrapper class `T` was generated automatically by `cjc`.

(**) `T'` must be either a mirror type, an interop class, or one of the value types listed above, e.g. `Int32`.

Use `Unit` as a return type to call a `void` Java method.

Limitations:

- Variable arity (varargs) methods and constructors are not supported.

Step 0: Build your Android application normally

Step 1: Generate mirror type declarations

Skip this step if you only want to call member functions of interop classes, their superclasses and/or types for which you have already generated mirror declarations, such as the types of interop class parameters or return values.

Command line:

```
java-mirror-gen \
    --package-name <package-name> \
    --class-path <full-application-classpath> \
    --destination <output-directory> \
    <names-of-mirrored-types>
```

or

```
/path/to/jdk/21/bin/java \
java-mirror-gen \
    --package-name <package-name> \
    --class-path <full-application-classpath> \
    --destination <output-directory> \
    -jar <jar-file>
```

where

- `<package-name>` is the desired name of the Cangjie package for all mirrored types.
- `<full-application-classpath>` is the complete classpath of the application that is used during its build, including `android.jar`.
- `<output directory>` is the pathname of the directory where the Cangjie source files containing mirror type declarations will be placed, such as `./src/cj`.
- `<names-of-mirrored-types>` are the *fully qualified names* of the Java reference types on which your interop class design depends, other than `java.lang.Object`, `java.lang.String` or arrays.
- `<jar-file>` is the pathname of a single jar file. All `public` classes and interfaces, the `.class` files of which it contains, are mirrored, along with any dependencies found along the `<full-application-classpath>`.

Example:

Continuing the example from the previous section, suppose we want our function `Interop.m()` to call the method `String g(A a, int i)` of a class

`com.example.c.C`, where `A` is the type `com.example.a.A` that we've already mirrored:

```
package com.example.c;

import com.example.a.A;

public class C {
    public static String g(A a, int i) {
        /* Some Java code returning a string */
    }
}
```

We re-run the mirror generator, passing `com.example.c.C` as an extra argument:

```
java-mirror-gen \
    --package-name javaworld \
    --class-path /home/user/Android/Sdk/platforms/android-35/android.jar:App.jar \
    --destination ./src/cj \
    com.example.a.A com.example.b.B com.example.c.C
```

This command will generate the same files with mirror type declaration plus the file `src/javaworld/src/C.cj` and files with mirror type declarations for any dependencies that type may have that were not previously mirrored.

The file `src/cj/javaworld/src/C.cj` would look like this:

```
package javaworld

import java.lang.*

@JavaMirror["com.example.c.C"]
public class C {
    public static func g(a: ?A, i: Int32): ?JString
}
```

Step 2: Import the mirror type(s) and write code

Make sure that all mirror type(s) involved are imported, then implement the constructors and member functions of your interop classes, manipulating Java types as if they were Cangjie types.

Example:

Continuing the example from the previous section, we can implement the function `cjworld.Interop.m()` using the mirror type `javaworld.C`:

```
package cjworld

import java.lang.*
```

```
import javaworld.A
import javaworld.B
import javaworld.C

@JavaImpl
public class Interop {
    public static func m(a: ?A, s: ?JString, i: Int32): ?B {
        let s1: JString = match (a) {
            case Some(aa) => C.g(aa, i) ?? JString("")
            case None => JString("")
        }
        B(s1)  // Assuming there is a B(String) constructor
    }
}
```

Step 3: Recompile the Cangjie part

See Step 4 of the interop class creation workflow for details and example.

As long as you have not changed the public interface of the interop class(es) on the previous steps, you only need to update the `.so` file. The generated Java sources should be identical to previously generated ones.

Step 4: Update and re-build the Android project

Copy the `.so` file and, if applicable, `.java` files updated/generated on the previous step over to your Android project and re-build it.

See Step 5 of the interop class creation workflow for details and example.

Features and Limitations of Interop Classes

1. An interop class *must* be a direct subclass of a mirror class. By default, the interop class will inherit the mirror class `java.lang.JObject`, not `std.core.Object`.
2. An interop class *may* implement one or more mirror interfaces, but never a conventional Cangjie interface. Conversely, a conventional Cangjie type may not implement or inherit a mirrored Java interface.
3. An interop class may not be declared as `open` or `abstract` and may not be extended using `extend`.
4. An interop class may introduce new instance fields *of any Cangjie type* and override the member functions of its mirrored superclass.
5. The constructors of an interop class may call superclass constructors using `super()`, but have the same limitation on instance member function calls

as the conventional Cangjie constructors, as well as the requirement to initialize all newly introduced member variables.

6. The instance member functions of an interop class may call instance member functions which that interop class has inherited from its mirrored superclass, and/or use `super.` to call the member functions of that superclass that the interop class overrides.

7. The signatures of constructors and member functions of all mirror types and interop classes can only use types that are (a) mirror types or interop classes themselves, or (b) are 100% analogous to Java primitive types. See the Type Mapping table in Calling Java from Cangjie.

8. Values of mirror types and interop classes:

   - Must not escape to global or static Cangjie variables, nor to any data structures to which such variables refer. The cjc compiler does not enforce this restriction yet, so strict programming discipline is required. Failure to comply with that discipline may lead to an abnormal program termination.
   - Must be explicitly disposed of when no longer needed. Best if at first opportunity, but in any case before control returns to the JVM. Failure to do so may result in memory leaks and the JVM throwing an `OutOfMemoryError`.

   See the section Java Reference Management for details.

The following three limitations are Android/JVM specific:

9. Any Cangjie code that uses a mirror type or interop class type must be executed in a thread registered in the Java VM. That may be a thread created in the Java code or an O/S thread registered in (aka "attached to") the JVM using the Java Invocation API. This is another requirement that must be met through programming discipline and obedience in the current version, but may be enforced by the Cangjie compiler in the future.

10. The Java counterparts of all mirror types and interop classes must all be loaded by the same classloader.

11. Unlike Java and other JVM languages, Cangjie does not permit circular import dependencies to exist between packages. That poses a challenge to the mirror generation process. Refer to the Circular Import Dependencies Handling subsection for details.

Java to Cangjie Mapping

The current version of the mirror generator does the following Java → Cangjie conversions.

General Considerations

The mirror generator takes Java *class* files as input. Any information that the Java compiler does not propagate from Java source code to class files is therefore not available to the mirror generator. Most notably, this concerns generics and names of method parameters.

Names

The original names of Java types, fields and methods are preserved to the maximum possible extent. If the original name could not be preserved for one of the reasons outlined below, it is propagated to the Cangjie compiler via an annotation.

- Java identifiers that clash with Cangjie keywords, such as "`func`", "`main`" or `Int32` are enclosed in backticks `` to form raw Cangjie identifiers:

  ```
  public static final long Int32 = 0xffff_ffff;
  ```

  ```
  public static let `Int32`: Int64
  ```

- Java identifiers may contain characters that are not permitted in Cangjie identifiers, most notably the dollar sign `$`, which is used in binary names of nested Java types. Such characters are replaced with underscores `_`:

  ```
  public class Outer {
      public class Inner {}
      public Inner getInner() { return new Inner(); }
  }
  ```

  ```
  @JavaMirror["Outer"]
  public open class Outer {
      public init()

      public open func getInner(): ?Outer_Inner
  }

  @JavaMirror["Outer$Inner"]
  public open class Outer_Inner {
      public init(p0: ?Outer)
  }
  ```

- Although it is considered bad practice, a Java type may have a field, a member type, and a method share the same simple name (identifier). This works, because it is always possible to determine the meaning of a name either by syntax (only method names can be followed by "`(`") or from context. In addition, instance and static methods of a type may have the same names in Java as long as their signatures are different.

  In Cangjie, however, all member variables and non-overloaded functions

declared in a given scope must have distinct names. (There are no member types in Cangjie, so name clashes with member types simply never happen, as they are mirrored into top-level types. In particular, an instance member function and a static member function cannot have the same name.

The mirror generator therefore appends "*_type-name*" to the names of instance variables and "`Static`" to the names of `static` methods if their original Java names clash with other field/method names. It retains the original name as the value of the `@ForeignName` annotation:

```
public class Node {
    public int id;
    public Node(int id) { this.id = id; }
    public static int id(long x) { return (int)x; }
    public static int id(short x) { return x; }
    public int id() { return id; }
    public void id(int newId) { this.id = newId; }
}
```

translates to

```
public open class Node {
    @ForeignName["id"]
    public var id_Node: Int32

    public init(arg0: Int32)

    @ForeignName["id"]
    public static func idStatic(arg0: Int64): Int32

    @ForeignName["id"]
    public static func idStatic(arg0: Int16): Int32

    public open func id(): Int32

    public open func id(arg0: Int32): Unit
}
```

- Names of Java packages cannot be preserved in practice as circular import dependencies between packages, omnipresent in Java, are forbidden in Cangjie. See Circular Import Dependencies Handling for details.

Primitive Types

Java primitive types are mirrored to the respective Cangjie value types:

Java Type	Cangjie Type
`boolean`	`Bool`
`byte`	`Int8`
`short`	`Int16`
`char`	`UInt16`
`int`	`Int32`
`long`	`Int64`
`float`	`Float32`
`double`	`Float64`

Classes and Interfaces

Java class and interface *definitions* are mirrored respectively into Cangjie class and interface declarations annotated with `@JavaMirror`. The value of that annotation is a string that retains the original fully qualified name of the mirrored Java type. Simple names of mirrored types are preserved unless they contain characters that are not permitted in Cangjie identifiers. Such characters, most notably the dollar sign `$`, the presence of which in a Java class file name is usually an indication of the respective type being a nested type, are replaced with underscores `_`.

Uses of Java class and interface types as types of mirrored fields, and/or as types of parameters and return values of mirrored methods and constructors, get wrapped in `Option<T>` (see Null Handling for details.)

`@JavaMirror`-annotated declarations differ from conventional Cangjie class/interface definitions in a few aspects:

- The root of the Java mirror classes hierarchy is not `std.core.Object`, but a built-in mirror class `java.lang.JObject`
- The mirror for `java.lang.String` is also built-in, its name is `java.lang.JString`
- Only the symbolic information is mirrored. Variable initializers and function/constructor bodies are omitted.

Example:

For example, a mirror class for the following Java class:

```
public class Node {
    public static final int A = 0xDeadBeef;
    private int id;
    public Node(int id) { this.id = id; }
    public int id() { return id; }
}
```

might look like this:

```
@JavaMirror["Node"]
public open class Node {
    public static let A: Int32
    public init(id: Int32)
    public func id(): Int32
}
```

Only public Java classes and interfaces are mirrored. Non-`final` classes are modified with `open`. The modifiers `sealed` and `non-sealed` are ignored, as well as the legacy modifier `strictfp`.

Private and package-private members and constructors, as well as static and instance initializers, are not mirrored.

If the names of a Java type member and its mirror differ for one of the reasons outlined in Names the original Java member name is propagated to Cangjie via the value of the `@ForeignName` annotation on the mirror member:

```
CurrencyAmount priceInUS$Per(WeightUnit wu) { ... }

@ForeignName["priceInUS$Per"]
public open priceInUS_Per(arg0: WeightUnit): CurrencyAmount
```

> **NOTE:** The meaning of the access modifier `protected` is different in Java and Cangjie.
>
> In Java, access to a `protected` member or constructor of a class is permitted from anywhere within the same package and from subclassses defined in other packages.
>
> In Cangjie, that modifier also permits access from subpackages and from any package within the same module.
>
> Normally, this should not pose a problem.

Fields are mirrored into member variables of the respective mirror types. Field names are preserved as long as they don't clash with names of other members. The modifiers `public`, `protected` and `static` are preserved. The modifiers `transient` and `volatile` are ignored. `final` Java fields are mirrored into `let` member variables, non-`final` — into `var` member variables. Variable initializers are omitted.

Methods are mirrored into member functions with the respective mirror types substituted for parameter types and return value type. Mirrors of `void` methods have the `Unit` return type. Instance method names are preserved. Static method names are reserved as long as they don't clash with instance method names. The modifiers `public`, `protected` and `static` are preserved. The modifiers `native` and `synchronized` are ignored, as well as the legacy modifier `strictfp`. Mirrors of non-`final` methods are modified with `open`.

Constructors are mirrored into `init` constructors with the respective mirror types substituted for parameter types. *This includes the default constructor that*

might have been implicitly declared. The modifiers `public` and `protected` are preserved.

NOTES:

1. Mirror classes may not have primary constructors.
2. Mirror classes that contain no `init` constructors do *not* get a default one defined implicitly, unlike the conventional Cangjie classes, and thus cannot be instantiated by calling a constructor. For automatically generated mirror classes, the absence of a constructor usually means that the mirrored Java class declares only `private` or package-private constructors so as to prevent its instantiation by arbitrary code. See also Enum Classes.
3. The Java mirror generator takes *class files* as input, and the names of method/constructor parameters are often absent in class files. In that case, the mirror generator will assign them names `arg0`, `arg1`, and so on. The `javac` compiler option `-parameters` forces parameter names retention, but only for classes, not interfaces. Same goes for the debug information generation option `-g` / `-g:vars`.

Member types are mirrored into the respective top-level types, as Cangjie does not support type nesting. The name of a member type mirror is derived from the *binary* name of the original member type, which consists of the binary name of its immediately enclosing type, followed by a dollar sign `$`, followed by the simple name of the member type itself. However, the dollar sign is not permitted in Cangjie identifiers, so it is replaced with an underscore `_` (see also Names). The modifiers `public` and `protected` are preserved. The modifier `static` is ignored. `init` constructors of mirrored inner classes have an extra parameter for passing the enclosing instance (that parameter is implicit in Java):

```
public class Outer {
    public static class Static {}
    public class Inner {}
    public Inner getInner() { return new Inner(); }
}
```

```
@JavaMirror["Outer"]
public open class Outer {
    public init()

    public open func getInner(): ?Outer_Inner
}

@JavaMirror["Outer$Static"]       // Original binary name is retained
public open class Outer_Static {  // '$' is replaced with '_'
    public init()
}
```

```
@JavaMirror["Outer$Inner"]       // Original binary name is retained
public open class Outer_Inner {  // '$' is replaced with '_'
    public init(p0: ?Outer)      // Extra parameter for enclosing instance
}
```

Bodies are omitted in the mirrors of *all* methods and constructors, so they all look like abstract member functions in regular Cangjie code. That imposes the following syntax alterations:

- The modifier `abstract` on *class* methods is preserved, as otherwise there would have been no way to tell apart the mirrors of abstract and concrete Java methods:

  ```
  public abstract class A {
      public void c() {}
      public abstract void a();
  }
  ```

  ```
  @JavaMirror["A"]
  public abstract class A {
      public init()

      public open func c(): Unit

      public open abstract func a(): Unit
  }
  ```

- For the same reason, mirrors of default interface methods are annotated with `@JavaHasDefault`.

  ```
  public interface I {
      default void c() {}
      void a();
  }
  ```

  ```
  @JavaMirror["I"]
  public interface I {
      @JavaHasDefault
      func c(): Unit

      func a(): Unit
  }
  ```

In methods with a variable number of parameters, `, ...` is ignored.

Mirror Type Inheritance

Mirror classes and interfaces form separate subtype hierarchies, which means that:

- The root of the Java mirror classes hierarchy is not `std.core.Object`, but a built-in mirror class `java.lang.JObject`.
- Mirror interfaces may inherit other mirror interfaces, reflecting the inheritance relationships between the original Java interfaces. Mirror interfaces may not inherit regular Cangjie interfaces and vice versa.
- Mirror classes may inherit other mirror classes, reflecting the inheritance relationships between the original Java classes. Mirror classes may not inherit regular Cangjie classes and vice versa.
- Mirror classes may implement mirror interfaces, but not regular Cangjie interfaces. In particular, they do not implement the interface `Any`. Regular Cangjie classes may not implement mirror interfaces.

Generics

Java generics are erased during compilation, so the automatically generated mirror types are always non-parameterized and have type variables replaced with mirrors of their leftmost bounds.

It is not possible to declare a parameterized mirror type manually either.

The type `JArray<T>` (see Arrays) is a special exception.

Arrays

The special built-in type `java.lang.JArray<T>` is used to represent mirrors of Java arrays.

A Java array of type *T* (*T*`[]`), is generally mirrored into:

- `?JArray<`*T'*`>`, if *T* is a primitive type
- `?JArray<`*?T'*`>`, if *T* is a reference type

where *T'* is the mirror of *T*. Refer to Null Handling for the reasoning behind `Option<T>` wrapping.

NOTE: Java arrays are covariant, whereas Cangjie generics are invariant, and the class `JArray<T>` is no exception.

Enum Classes

A Java enum class *E* is mirrored into a Cangjie mirror class *E'* that inherits the mirror of `java.lang.Enum` and is neither `open` nor `sealed` itself, thus cannot be extended.

The class *E'* contains:

- Mirrors of the enum constants of *E*, in the form of public static member `let`-variables of type *E'*.

- No constructors, for the instantiation of the mirror class to be impossible. (Mirror classes do not have default constructors.)
- Mirrors of the implicitly defined methods `public static E[] values()` and `public static E valueOf(String name)`
- Mirrors of any `public`/`protected` fields and methods of the original enum class as if they were fields and methods of an ordinary class

Special Considerations

`null` Handling

Cangjie has no concept of null references and hence no equivalent for the Java null type. If the reference type of a Java method return value was mirrored direct to the respective mirror type, any `null` value returned from that method could lead to a segmentation fault. Conversely, there has to be a way to pass a `null` value from Cangjie to a Java method or constructor.

Therefore, if the original Java type of a field, array element, method/constructor parameter or method return value is a reference type *R*, the mirror of that *entity* will have the type `Option<R'>`, where *R'* is the mirror of the *type* *R*. On the Cangjie side, `None` stands for the `null` value, and `Some(r')` represents a (non-null) reference value *r'* of the type *R'*. The `cjc` compiler recognizes `Option<T>` as a Java-compatible type if `T` is a mirror type or interop class and wraps/unwraps the values of `T` accordingly.

For instance, consider the following Java interface:

```
interface Concatenator {
    String concat(String[] ss);
}
```

The `ss` parameter itself may be `null`, each element of the `ss` array may be `null` and the `concat` method may return `null`. Therefore, the *safe* way to mirror that interface is as follows:

```
@JavaMirror
interface Concatenator {
    func concat(ss: ?JArray<?JString>): ?JString
}
```

For the same reason, you should use `Option<T>` wrapping when declaring a local variable of a foreign type in the code of an interop class, unless you are 100% sure that it won't be assigned `null`.

```
    // Suppose M is a Java mirror type
    let m: M = M()     // If M() returns successfully, it is guaranteed
                       // to have returned a new instance of M
```

The `Option<T>` wrapping ensures that the code won't break if a `null` value sneaks into the Cangjie world from the Java one, but it does that at the cost of performance and memory footprint. The other disadvantage of this approach is the loss of variance.

Loss of Variance

One limitation imposed by the `Option<T>` wrapping of Java mirror types and interop classes is that such wrapped types follow the semantics of Cangjie in all other respects. In particular, `Option<T>` is *invariant by its type parameter* `T`: `Option<U>` is not a subtype of `Option<T>` if `U` is a subtype of `T`, unless `U` and `T` are the same type. For mirror types that means that any overriding Java method that relies on return type covariance may not be mirrored that way with `Option<T>` wrapping. Its return type has to be propagated from the superclass method.

Example:

Suppose a Java class `Foo` is a direct superclass of the class `Bar`:

```
public class Foo {}

public class Bar extends Foo {}
```

and the interface `C` declares a method `get` that returns an instance of `Foo`:

```
public interface C {
    public Foo get();
}
```

A subinterface of `C` may then override `get` with a more precise return type, `Bar`:

```
public interface D extends C {
    @Override
    public Bar get();
}
```

Without `Option<T>` wrapping, all those types could be mirrored to:

```
@JavaMirror
public open class Foo {}

@JavaMirror
public open class Bar <: Foo {}

@JavaMirror
public interface C {
    public open func get(): Foo
}
```

```
@JavaMirror
public interface D <: C {
    public override open func get(): Bar       // Return type covariance in action
}
```

but if `get()` can possibly return `null`, an application crash is inevitable.

`Option<T>` wrapping makes it safe, but the return types of all overriding methods have to be lowered to the return type of the original method:

```
@JavaMirror
public open class Foo {}

@JavaMirror
public open class Bar <: Foo {}

@JavaMirror
public open interface C {
    public open func get(): ?Foo
}

@JavaMirror
public open interface D <: C {
    // public open func get(): ?Bar    // Error, `Option<T>` is not covariant by T
    public open func get(): ?Foo       // OK, but the return type is lowered
}
```

Nullability annotations could partially alleviate the problem, but the current version of the Cangjie SDK does not support them.

Circular Import Dependencies Handling

Circular dependencies between packages are not only possible, but omnipresent in Java. For instance, the ubiquitous class `java.lang.String` depends on:

- the interface `Serializable` from the `java.io` package,
- the class `Charset` from `java.nio.charset`, and
- the class `Locale` from `java.util`,

and all those three types of course depend on the class `Object` from `java.lang`!

Compilation to separate `.class` files and late binding effectively circumvent this problem in Java. For a Cangjie import declaration to compile, however, the entire package that it imports (from) must be already compiled. Consequently, all Cangjie source files that belong to a given package must be compiled together in one session into a single binary. One can say that the minimum unit of compilation in Cangjie is an entire package, not an individual file. Hence circular import dependencies between packages are not possible in Cangjie code.

Single-Package Mode

The above difference between the two languages precludes the preservation of Java package names during mirror generation. In practical terms, the mirror generator must *always* collect all generated mirror types in a single Cangjie package. The desired name of that package must be specified with the option `--package-name`.

For instance, if you run the mirror generator with the following option:

```
--package-name java.world
```

it will place all generated mirror type declarations in the `java.world` package and propagate the fully qualified names of the original Java types to the `cjc` compiler via arguments of the `@JavaMirror` annotations:

```
package java.world

import java.lang.*

@JavaMirror["java.lang.Cloneable"]
public interface Cloneable {
}
```

Name Clashes

Java types declared in different packages may have the same simple name, because their fully qualified names are different. For example, there is a class `Attribute` in the JDK package `javax.management` and an interface `Attribute` in `javax.naming.directory`. Their mirrors cannot co-exist in a single Cangjie package. Therefore, the mirror generator detects such name clashes when operating in the single-package mode and mangles *all* conflicting names. Specifically, it uses the fully qualified name of each such type in place of its simple name, replacing dots `.` with underscores `_`:

```
// src/java/world/src/javax_management_Attribute.cj
package java.world

import java.lang.*

@JavaMirror["javax.management.Attribute"]
public open class javax_management_Attribute <: Serializable {
   .  .  .
```

```
// src/java/world/src/javax_naming_directory_Attribute.cj
package java.world

import java.lang.*
```

```
@JavaMirror["javax.naming.directory.Attribute"]
public interface javax_naming_directory_Attribute <: Cloneable & Serializable {
   .  .  .
```

Incremental Mirroring

Putting all Java mirror types into a single Cangjie package such as `java.world` not only leads to name clashes, but also increases the compile-time overheads and is generally inconvenient. Breaking that mass of declarations down into several packages is therefore highly desirable. Fortunately, not all Java packages are in a single import dependency circle, so at least some breaking down is possible.

First of all, you may notice that there are three sets of packages in any Android application that may not have circular dependencies between them:

- Application packages
- Android API packages
- JDK API packages

This means that even in the worst possible case all mirror types could be segregated into three Cangjie packages, respectively code-named "`app`", "`android`" and "`java`".

Furthermore, the JDK API has been modularized since Java 9, and circular import dependencies may *not* exist between Java modules. Therefore it is perfectly possible to replicate the structure of the JDK API *modules* during mirroring. For instance, the Cangjie package `java.base` would then contain mirrors of types from the packages exported from the `java.base` JDK API module: `java.lang` and its subpackages, `java.io`, `java.math`, and so on.

Finally, a third party library should have no circular import dependencies with other application components and APIs. If your application contains any such libraries that you also need to use from Cangjie code, you may want to mirror each library into its own Cangjie package.

The *incremental mirroring* process enables such segregation. You run the mirror generator *once for each target Cangjie package* (set with the `--package-name` option, as usual), each time specifying two additional parameters:

1. A *complete* list of Java packages to mirror into that specific target Cangjie package. Those Java packages may have circular import dependencies between them, and may also import *previously* mirrored packages/types.
2. A cumulative list of fully qualified name mappings for all *previously* mirrored Java types. Initially this list is empty.

The mirror generator emits mirrors for all public classes and interfaces that belong to the packages appearing on the list (1), and any dependencies they may

have that are *not* already on the list (2). Then it appends all newly established Java → Cangjie name mappings to the list (2).

Command-line Syntax Incremental mirroring is only supported in the single-jar mode at present, which means that the second variant of the mirror generator command-line syntax has to be used:

`java-mirror-gen [`*`options`*`] -jar `*`jar-file`*

`options` must include the following:

`--package-name `*`target-package-name`*

`target-package-name` is the name of the target Cangjie package. The mirror generator will place *all* mirror types into that package.

IMPORTANT: You must specify a new, previously unused target Cangjie package name every time you run the mirror generator in the incremental mode. If you mirror some types into, say, `my.java.libs` package in the first run of the mirror generator using `--package-name my.java.libs` and run it again against different input but without changing the value of `--package-name`, that would completely invalidate the results of mirroring, making them inconsistent.

> In a certain sense, a target Cangjie package is the minimal unit of mirroring, just like a source Cangjie package is the minimal unit of compilation when you develop in Cangjie: you cannot compile two Cangjie source files belonging to the same package separately.

`--package-list `*`pathname`*

`pathname` must point to a plain text file that contains a list of Java package names. Specifically, each line of that file must be a fully qualified package name, optionally followed by a wildcard `.*`:

```
com.example.model
com.example.ui.*
   .   .   .
```

NOTICE: If the wildcard is present, that entry matches all subpackages of the named package *and* that package itself. This is different from the semantics of the wildcard in Java and Cangjie import statements.

The generator will emit mirrors for *all* non-anonymous non-private types defined in the listed packages *and* all their dependencies *except* for the dependencies that are already present in the import mappings file, if any. All those mirror types will be placed in a single Cangjie package specified using the `--package-name` option (see above).

This option may only be used in the single-jar mode.

`--import-mappings `*`import-mappings-file`*

`import-mappings-file` is the pathname of the *import mappings file*, a plain text file that accumulates a set of Java to Cangjie fully qualified name mappings for all mirror types as they are generated. The mirror generator reads that file at startup and does *not* mirror the Java types for which mappings already exist, assuming that they were mirrored on its prior invocation(s). If mirroring completes without errors, the mirror generator adds the mappings for all mirrors it has just generated to the set and writes it out to the `./imports_config.txt` file before termination.

NOTE: If *`import-mappings-file`* is set to `./imports_config.txt`, that file will get overwritten. Make a copy after each incremental run of the mirror generator if you want to keep the intermediate sets of mappings, e.g. for debugging purposes.

Example: JDK API Mirroring

As mentioned above, the JDK API is modularized, so one could look at the list of modules and their dependency graphs, collect lists of exported packages, and run the mirror generator using the following command line template:

```
java-mirror-gen \
    --android-jar /path/to/android.jar \
    --package-name java.base \
    --package-list ./java.base.txt \
    --import-mappings ./imports_config.txt \
    --destination src/cj \
    --class-path /path/to/android/SDK/platform/files/directory \
    -jar /path/to/android.jar
```

The file `./java.base.txt` should contain the list of packages exported from the `java.base` JDK API module *for public use*:

```
java.io
java.lang
java.lang.annotation
   .   .   .
```

and the file `./imports_config.txt` should be empty or non-existent.

Upon the successful completion of the above command:

- The directory `./src/cj/java/base/src` will contain Cangjie source files with mirror type declarations for all public classes and interfaces defined in the packages listed in `./java.base.txt`, and
- The file `./imports_config.txt` will contain mappings of fully qualified names of all those Java types to the names of their Cangjie mirrors in the `java.base` package, for use in subsequent mirror generator invocations.

> **NOTICE:** The *implementations* of many types exported from the module `java.base` depend on classes and interfaces defined in internal JDK packages, such as `sun.util.locale.provider.LocaleDataMetaInfo` or `jdk.internal.misc.Unsafe`. However, mirror types are *declarations*; they do not expose any private members of the respective Java types nor the implementation details of their public methods or constructors. So the mirror generator will not process the class files that belong to such internal packages at all.

Now we can pick a JDK API module that depends only on the `java.base` module, e.g. `java.xml`, and run the mirror generator again, changing only the values of two options:

- Set the value of `--package-name` to `java.xml`
- Set the value of `--package-list` to the pathname of a file containing the list of packages exported from the `java.xml` JDK API module:

  ```
  javax.xml.*
  org.w3c.dom
     .  .  .
  ```

Note that it is possible to put a wildcard `.*` after a package name to denote that package *and* all its subpackages at once. In the above excerpt, `javax.xml.*` matches the package `javax.xml` itself, as well as `javax.xml.catalog`, `javax.xml.transform`, `javax.xml.transform.stream`, and so on.

```
java-mirror-gen \
    --android-jar /path/to/android.jar \
    --package-name java.xml \
    --package-list ./java.xml.txt \
    --import-mappings ./imports_config.txt \
    --destination src/cj \
    --class-path /path/to/android/SDK/platform/files/directory \
    -jar /path/to/android.jar
```

`./src/cj/java/xml/src` should now contain mirror declarations for all `java.xml` module exports, whereas `./imports_config.txt` should now contain mappings for *both* `java.base` and `java.xml` Cangjie packages.

Repeat this process for the remaining modules, in any order that ensures mirroring of all dependencies of each module before mirroring that module itself.

Using Incremental Mirroring

You can use incremental mirroring to segregate arbitrary components of the application code that need to be used from Cangjie, as long as there are no circular import dependencies between those components and/or the rest of the Java code. In particular, the public classes and interfaces of each third party

Java library that Cangjie code needs to use can be mirrored separately into an appropriately named Cangjie package.

To accomplish that, first break down the set of packages constituting the application into subsets that have no circular import dependencies between them. Then use the incremental mirroring process described above:

1. Pick any subset that has no import dependencies on packages from other subsets. Mirror it all at once into an appropriately named package.
2. Mirror any subset that only depends on zero or more of the previously mirrored subsets.
3. Repeat the previous step until all Java types that you want to use in Cangjie code are mirrored, *each time picking a new target Cangjie package name*.

Closure Depth Limiting

In addition to numerous circular import dependencies between packages, another property is inherent to the standard Java API and many popular Java libraries and frameworks: enormous sizes of import closures.

For instance, invoking the mirror generator to produce a mirror for an empty enum type

```
public enum E {}
```

makes it also generate mirrors for `java.lang.Enum` *and all its dependencies* from the standard Java library, about 300 mirror types total:

```
AbstractInterruptibleChannel.cj
AbstractStringBuilder.cj
AccessControlContext.cj
AccessMode.cj
AccessibleObject.cj
   .  .  .
ZoneOffsetTransitionRule.cj
ZoneRules.cj
ZonedDateTime.cj
```

For all those mirror types, the compiler will generate glue code, most of which a real program will never use. That may be tolerated in a desktop or server environment, but not on a mobile platform such as Android.

Asking the developer to supply an exact list of types and members to mirror would be too much. Fortunately, the simple *limiting of depth* during the calculation of dependency closures yields reasonably good results: the number of generated mirrors can drop by an order of magnitude or more. For instance, mirroring the above empty enum type `E` with depth limited to 2 produces just six mirror types in

addition to the mirror of the type `E` itself: the enum base class `java.lang.Enum`, three interfaces that it implements, and the return types of two of its methods:

```
Class.cj
Comparable.cj
Constable.cj
E.cj
Enum.cj
Optional.cj
Serializable.cj
```

Here is how depth limiting works exactly:

- The set of mirrored *types* always includes:
  - Primitive Java types, mirrored into the respective Cangjie value types;
  - Class and interface types explicitly specified on the mirror generator command line;
  - `java.lang.Object` and `java.lang.String`, pre-mirrored respectively into `JObject` and `JString`; and
  - Array types, the element types of which are among the above listed types, mirrored into `JArray<T>`.
- *Members* of types included in the set are then filtered as follows:
  - Inherited members are *not* mirrored as "own" members even if the respective supertypes are not included in the set of mirrored types.
  - Fields, the types of which are not included in the set of mirrored types, are omitted.
  - Methods, the signatures of which contain types that are not included in the set of mirrored types, are omitted.
  - Member types are treated just like top-level types: unless they are explicitly specified on the mirror generator command line, they only get mirrored if included in the set of mirrored types during the limited-depth dependency closure calculation process described here.
- The command-line option `--closure-depth-limit` sets the depth limit for the explicitly specified class and interface types.
- If the depth limit for a given type *`T`* is zero, *no types on which `T` depends get added to the set*. Even the supertypes of *`T`* are not added.

  Example:

  ```
  // A.java
  public class A {
      public void f(String s) {}
  }
  ```

```java
// B.java
public class B extends A {
    public void g(String s) {}
}
```

If the mirror generator is instructed to mirror the class `B` with depth limited to 0, it will only generate a mirror for `B` and its method `g(String)`, because `JString` is automatically included in the set of mirrored types. At the same time, the method `f(String)` that `B` inherits from `A` will not be available:

```
// B.cj
   .  .  .
@JavaMirror["B"]
public open class B {
    public init()

    public open func g(s: ?JString): Unit
}
```

- If the depth limit for a given type *`T`* is any positive *`N`*, the set of mirrors *additionally* includes the following types, with the depth limit of *`N`*`-1`:
  - All (that is, recursively collected) supertypes of *`T`*;
  - Types of the non-`private` fields *declared in `T` itself*, that is, not the types of inherited fields;
  - Types of parameters of the non-`private` constructors of *`T`*; and
  - Types of parameters and return values of the non-`private` methods *declared in `T` itself*. Note again that the inherited methods are *not* scanned.

  If a type is already included in the set, but with a lower depth limit, the latter is changed to *`N`*`-1`.

  The process is repeated recursively for all types newly added to the set of mirrors.

  Example:

```java
// A.java
public class A {
    public void f(C c) {}
}

// B.java
public class B extends A {
    public void g(D d) { }
}

// C.java
```

```
public class C { }

// D.java
public class D extends C {}
```

If the mirror generator is instructed to mirror the class `B` with depth limited to 1, it will generate mirrors for `B`, its superclass `A` and the class `D` that is the type of the sole `B.g(D)` method parameter. However, `A` and `D` will be mirrored with depth limit set to 0, so the class `C`, and hence the method `A.f(C)` will not be mirrored:

```
// A.cj
   .  .  .
@JavaMirror["A"]
public open class A {
    public init()
}

// B.cj
   .  .  .
@JavaMirror["B"]
public open class B <: A {
    public init()

    public open func g(d: ?D): Unit
}

// D.cj
   .  .  .
@JavaMirror["D"]
public open class D {
    public init()
}
```

Finally, setting the depth limit to 2 will make the mirror generator include mirrors of the class `C` and the method `A.f(C)` in its output.

Java Mirror Generator Reference

Prerequisites

The Java mirror generator requires a copy of JDK 17 to run. If you do not have JDK 17 installed and added to your executable search `PATH` install and/or add it before using the Java mirror generator.

You also need to know the *local* pathnames of the jar files and directories that contain all dependencies of the classes you are going to mirror. This includes the

Android standard library jar as well as the jars that will appear on the classpath when your application launches.

Command-line Syntax

The Java mirror generator can be run in two ways:

`java-mirror-gen [`*`options`*`] [`*`type-names`*`]`

to generate mirrors for one or more Java classes and/or interfaces *and* all their dependencies, *or*

`java-mirror-gen [`*`options`*`] -jar `*`jar-file`* *(single-jar mode)*

to generate mirrors for all types the `.class` files of which the given *`jar-file`* contains *and* all their dependencies residing elsewhere on the supplied class path, if any,

where

- *`options`* are the command-line options of the mirror generator.
- *`type-names`* are the *fully qualified names* of Java classes and interfaces that need to be mirrored (that is, *not* pathnames of their `.class` files).
- *`jar-file`* is the pathname of a single jar file.

NOTE: (Incremental mirroring and/or closure depth limiting may restrict the set of mirrored types and/or their members during each invocation of the mirror generator.

Command-line Options

- `-a` *`pathname`*, `-android-jar` *`pathname`* *(mandatory)*

  `pathname` must point to the `android.jar` file from the Android SDK used to build the Java part of the application. This option must always be present and point to an existing file, otherwise the script shall fail with an error message.

- `-d` *`directory`*, `--destination` *`directory`*

  `directory` is the pathname of the directory into which the mirror generator shall place its output Cangjie source code files containing the generated mirror type declarations, in a hierarchy of subdirectories conforming to CJPM requirements. If this option is not specified, the current directory is assumed.

- `-cp` *`path`*, `--class-path` *`path`*

  `path` is a list of pathnames of directories, jar files and zip files, separated with semicolons `;` on Windows and colons `:` on all other platforms. The mirror generator shall go through those directories and archives when

looking for the class files of the Java types specified on the command line (*type-names*) *and* their dependencies.

- `-p` *name*, `--package-name` *name* *(mandatory?)*

  name is the name of the Cangjie package into which all generated mirror types will be placed. See Single-Package Mode for details.

- `-c` *number*, `--closure-depth-limit` *number*

  number is a non-negative decimal integer limiting the depth of dependency graph scanning when determining the set of types and their members that will be mirrored.

  See Closure Depth Limiting for details.

- `-jar` *jar-file*

  jar-file is the pathname of the jar file to be processed in the "single-jar" mode (see Command-Line Syntax above).

- `-l` *pathname*, `--package-list` *pathname*

  pathname is the pathname of a plain text file containing a list of package names. Only the class files that belong to the listed packages *and* their dependencies will be mirrored. Requires `-jar`. See Command-Line Syntax in the Incremental Mirroring section for details.

- `-i` *pathname*, `--imports` *pathname*

  pathname is the pathname of a plain text file containing mirror type mappings accumulated during previous mirror generator runs. Requires `-l` and `-jar`. See Command-Line Syntax in the Incremental Mirroring section for details.

- `-h`, `-?`, `--help`

  Prints a help message with a brief description of the command-line syntax and all options and exits.

- `-v`, `--verbose`

  Instructs the mirror generator to emit messages about what it is doing.

Examples

```
java-mirror-gen \
    -android-jar $ANDROID_SDK/platforms/android-35/android.jar \
    -class-path ./classes \
    -destination ./mirrors \
    -p com.example \
    com.example.subpkg1.A com.example.subpkg2.B
```

```
java-mirror-gen \
    -a $ANDROID_SDK/platforms/android-35/android.jar \
    -cp ./lib \
    -d ./mirrors \
    -p javaworld \
    -jar App.jar
```

For a more complex example, see Example: JDK API Mirroring.

Interop Library API Reference

The Android interop library includes enhanced versions of mirror types for two fundamental Java classes, `java.lang.Object` and `java.lang.String`, and a generic mirror type for representing Java arrays.

To enable the use of unqualified names, those mirror types are called respectively `JObject`, `JString`, and `JArray<T>` so as to avoid name clashes with Cangjie types from `std.core`.

They also have some methods renamed and new methods added to enhance interoperability.

`java.lang.JObject`

`java.lang.JObject` is the root of the hierarchy of Java mirror classes and interop classes. It is itself a mirror of the `java.lang.Object` class, but has some unsupported methods removed and some methods renamed and added for better alignment with the Cangjie standard library.

NOTE: The methods removed from `JObject` are not available in any other mirror classes either. Those are `clone()`, `finalize()`, and `getClass`.

```
package java.lang

@JavaMirror["java.lang.Object"]
open class JObject {
    public open func equals(obj: ?JObject): Bool

    public func hashCode(): Int64
    @ForeignName["hashCode"]
    public open func hashCode32(): Int32

    public func toString(): String
    @ForeignName["toString"]
    public open func toJString(): JString

    public func wait(timeoutMillis: Int64): Unit
    public func wait(timeoutMillis: Int64, nanos: Int32): Unit
```

```
    public func wait(): Unit
    public func notifyAll(): Unit
    public func notify(): Unit
}
```

The instance member function `equals` and all `wait/notify` functions are mirrors of the respective Java methods.

`public func hashCode(): Int64`

Calls the original Java method `hashCode` and casts the returned 32-bit `int` value to `Int64`. Introduced to better meet the expectations of Cangjie developers.

```
@ForeignName["hashCode"]
public open func hashCode32(): Int32
```

The original Java method `hashCode`, renamed to `hashCode32` to avoid a name clash.

`public func toString(): String`

Calls the original Java method `toString` and converts its result to a Cangjie `String`. In the *very* unlikely event of receiving `null` from the underlying Java method, throws an exception.

Introduced to better meet the expectations of Cangjie developers.

NOTE: This function returns a Cangjie `String`, breaking the requirement to use only Java-compatible types for parameters and return values of public member functions and constructors of mirror types and interop classes. This is only possible because the `cjc` compiler has dedicated support for this member function.

```
@ForeignName["toString"]
public open func toJString(): JString
```

The original Java method `toString`, renamed to `toJString` to avoid a name clash.

An implementation of Java `toString` is very unlikely to return `null`, hence the return type is `JString`, not `?JString`.

java.lang.JString

```
package java.lang

@JavaMirror["java.lang.String"]
open class JString {
        .  .  .
    public init(cjString: String)
        .  .  .
}
```

```
public init(cjString: String)
```

Effectively converts the given Cangjie `String` to a Java string (`JString`).

NOTE: This constructor accepts a parameter of the Cangjie type `String`, breaking the requirement to use only Java-compatible types for parameters and return values of non-private member functions and constructors of mirror types and interop classes. This is only possible because the `cjc` compiler has dedicated support for `JString`.

Members inherited from `JObject`: `equals`, `hashCode`, `hashCode32`, `toString`, `toJString`, `wait/notify` methods.

java.lang.JArray<T>

`JArray<T>` is a special built-in generic mirror type that mirrors all Java array types. `T` must be either a value type mapped to a primitive Java type, such as `Int32` or `Bool`, a mirror type, or an interop class.

Limitations:

As of the current version:

- Java arrays with nullable elements are not supported. In other words, if `T` is a mirror type or an interop class, the type `JArray<T>` is supported, but the type `JArray<?T>` is not.
- Variables and parameters of Java array types may not be marked as nullable using the `Option` enum. That is, even if `JArray<T>` is a valid type, `?JArray<T>` is not recognized as such.

The API of Java arrays is limited compared to that of the Cangjie `Array<T>` struct type; the only operations besides construction are length query, element access, and the inherited methods of `java.lang.Object`.

```
public init(length: Int32)
```

Constructs a new Java array of the given `length`.

```
public prop length: Int32
```

The number of elements in the array.

```
public operator func [](index: Int32): T
public operator func [](index: Int32, value!: T): Unit
```

The element access operator `[]`.

Members inherited from `JObject`: `equals`, `hashCode`, `hashCode32`, `toString`, `toJString`, `wait/notify` methods.

Introduction

The **Cangjie Multiplatform** technology enables Cangjie developers to incorporate their code into new Android/iOS applications and/or gradually replace the Java/Objective-C parts of existing applications with Cangjie code. The central mechanism that enables the cross-language, cross-runtime interoperability is *mirror types*, which expose the types of one language to the other.

On the Cangjie side, mirror types enable the inheritance of Java/Objective-C classes with method overriding, as well as the implementation of Java interfaces and Objective-C protocols, all while using conventional Cangjie syntax. On the Java/Objective-C side, the mirror types that represent Cangjie types are expressed in terms of the respective language's own types. All in all, this enables seamless transitions between the Java/Objective-C and Cangjie parts of the application, which includes usage of the target O/S APIs in Cangjie code.

The Challenge and the Solution

Java, Objective-C and Cangjie are object-oriented languages that support inheritance and polymorphism. However, the differences in their semantics, object models, and execution models preclude direct usage of Cangjie objects in Java or Objective-C code and vice versa.

Then, all three languages have managed runtimes that support automatic memory management, threading, exception handling and other low-level features, but again they do that differently. Making two separately designed complex language runtimes aware of each other might push the complexity of the entire system beyond human comprehension.

Instead, the interoperability between Cangjie and Java is achieved by disguising them both as low-level languages. Cangjie and Java parts of the application see each other through the lens of the Java Native Interface (JNI), originally designed to facilitate the development of Java `native` methods in languages such as C and C++. JNI is a rich, but low-level API, and writing `native` Java methods is known to be cumbersome. Fortunately, CJMP automates away the complexity of JNI.

The Objective-C Runtime module API serves a similar role on iOS. It is also designed specifically to enable the creation of bridge layers between Objective-C and other languages. And just like with JNI, CJMP takes care about the cumbersome parts.

Additional Setup

The current version of the Cangjie SDK for iOS requires you to download the `Cangjie.h` header file from the Cangjie open source repository:

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

and integrate it into your XCode project in order to use the interoperability features described in this document.

Key Concepts

Mirror Types

Considering Cangjie and Objective-C as a pair of interoperating languages, a *mirror type* *`T'`* defined in one language of the pair is a type that represents an existing type *`T`* defined in the other language, enabling the code written in the first language to use that type, possibly with some limitations.

The Boolean types and numeric types that are essentially the same in both languages naturally mirror each other: the Objective-C mirror type for the Cangjie type `Int32` is `int` and vice versa. Mirroring of numeric types that the other language does not naturally support is not currently possible, so e.g. the Cangjie type `Float16` cannot be mirrored to an Objective-C primitive type.

For a user-defined type such as class, struct, protocol, or interface, the mirror type would be its closest equivalent in the other language. For instance, the only Objective-C type that can approximate a Cangjie struct type with reasonable accuracy is an Objective-C class attributed with `objc_subclassing_restricted`.

A mirror type exposes the members and constructors of the original user-defined type that are accessible *and* can be used in the other language, so again a Cangjie function that returns a value of type `Float16` cannot be mirrored straight to Objective-C.

Normally you would obtain mirror type definitions for a type that you want to use in the other language *and* its required dependencies automatically. For Objective-C types, a standalone mirror generator is provided, whereas Objective-C mirrors of Cangjie types are generated by the `cjc` compiler as a byproduct of the normal compilation process.

Mirroring Objective-C Types to Cangjie

The `cjc` compiler replaces any uses of Objective-C mirror types with the appropriate glue code, so only the names of types themselves and names and types of their accessible members matter. Therefore, mirrors of Objective-C types only contain member declarations, not definitions: constructors and member functions/properties have no bodies and member variables have no initializers. For the same reason, `@private` members are omitted.

For such an extension of the regular Cangjie syntax to work, each mirror type must be marked with an `@ObjCMirror` annotation. It helps the compiler distinguish between mirror type declarations and normal Cangjie type definitions.

For example, a mirror class for the following Objective-C class:

```
@interface Node : NSObject {
}
- (id)initWith:(int)x;
- (int)getX;
@end
```

might look like this:

```
@ObjCMirror
public open class Node <: NSObject {
    @ForeignName["initWith:"]
    public init(x: Int32)
    public open func getX(): Int32
}
```

Mirroring Cangjie Types to Objective-C

Unlike the mirrors of Objective-C types, for which `cjc` has dedicated support, mirrors of Cangjie types must look exactly like normal Objective-C types from the outside, for the iOS toolchain to pick them up. The `cjc` compiler therefore generates those mirrors in the form of Objective-C source code files containing glue code responsible for linking the Objective-C and Cangjie worlds together.

For instance, if the class `Node` from the example in the previous section was originally defined in Cangjie as follows:

```
public class Node {
    private let _x: Int
    public func x(): Int { _x }
    public init(x: Int) { this._x = x }
}
```

the `cjc` compiler could generate the following Objective-C mirror for it (glue code mostly omitted for brevity):

```
// Node.h
@interface Node : NSObject
- (id)init:(int64_t)x;
- (int)x;
@end
```

```
// Node.m
@implementation Node
- (id)init:(int64_t)x {
    /* Glue code constructing a Cangjie Node instance and associating
     * it with the Objective-C Node instance being constructed, i.e. 'self'.
     */
}
- (int64_t)x {
```

```
    /* Glue code invoking the 'x' member function of the associated
     * Cangjie Node instance and returning the result.
     */
     }
@end
```

Mirror Functions

Both Objective-C and Cangjie support top-level, global functions that are not members of any other type. They are exposed to the other language as *mirror functions*, which are essentially automatically generated pieces of glue code that pass control and data through the inter-language barrier.

Interop Classes

An *interop class* is essentially a Cangjie class that is derived from one or more mirror types and is usable from Objective-C. All its constructors and non-inherited public member functions are exposed to Objective-C code via a conjugate *wrapper class*, automatically generated by the `cjc` compiler. The wrapper class itself defines no other user-callable methods or constructors, but any methods it may have inherited from its supertypes may be called from both Objective-C and Cangjie code.

For example, when compiling the following Objective-C interop class:

```
@ObjCImpl
public class BooleanNode <: Node {
    private let _flag: Bool
    public init(x: Int32, flag: Bool) {
        super.init(x)
        this._flag = flag
    }
    public func flag(): Bool {
        _flag
    }
}
```

the `cjc` compiler will also yield a pair of Objective-C source code files similar to the following:

```
// BooleandNode.h
@interface BooleanNode : Node
/* glue code */
- (id)init:(int32_t)x:(BOOL)flag;
- (BOOL)flag;
/* more glue code */
@end
```

```
// BooleandNode.m
@implementation BooleanNode : Node
/* glue code */
- (id)init:(int32_t)x:(BOOL)flag {
    /* Glue code constructing a Cangjie BooleanNode(x, flag) instance and
     * associating it with the Objective-C instance being constructed,
     * i.e. 'self'.
     */
}
- (BOOL)flag {
    /* Glue code invoking the 'flag' member function of the Cangjie
     * BooleanNode instance associated with 'self' and returning the result.
     */
}
/* more glue code */
@end
```

Foreign Types

The mirror types and interop classes are not perfectly native to the language in which they are defined. Extending the analogy, their status is more like temporary visitors and work visa holders respectively, so they are collectively called *foreign types* throughout this document.

Objective-C-Compatible Types

The following Cangjie types are called *Objective-C-compatible*:

- Value types that have direct equivalents among Objective-C primitive types (`Int16` is included, but `Float16` is not)
- Foreign types
- Types of the form `Option<T>` where *T* is a foreign type (see ***null** Handling* for reasoning)

For obvious reasons, the parameters and return values of public member functions of a foreign type may only have types compatible with the respective language.

The member variables of an interop class may have any types. Their public member variables that have Objective-C-compatible types may be accessed from both Objective-C and Cangjie.

Interop Scenarios

There are essentially two Objective-C-Cangjie interoperation scenarios:

- In one scenario, Cangjie types and functions are exposed to Objective-C, enabling the use of Cangjie APIs, libraries and application components in Objective-C code.

- In the other scenario, Objective-C classes and interfaces are exposed to Cangjie, enabling Cangjie code to access iOS APIs, third party Objective-C libraries and components of the application that remain coded in Objective-C.

The above scenarios are *not* mutually exclusive, one application may as well be utilizing both. Also, class inheritance and interface implementation are supported in both scenarios (an Objective-C class may subclass a Cangjie class and vice versa). That enables seamless callbacks, so neither scenario is unidirectional. However, each scenario has its own feature set, limitations, tooling, etc., so they are described separately.

Using Cangjie in Objective-C

To enable access to a Cangjie library, API, or some other piece of code from Objective-C, you need to generate Objective-C mirror types for the Cangjie types that you want to expose. The `cjc` compiler can do that for you as described in Cangjie Mirror Generation Reference.

NOTE: The type system of the Cangjie language is more versatile than its Objectiove-C counterpart. There are also some principal differences in the features shared by the two languages, most notably generics. Expressing certain features of Cangjie in Objective-C is therefore impossible, and other features can be difficult to express in a natural and compact manner. See Cangjie to Objective-C Mapping for the exact list of supported features and associated restrictions.

Example:

Suppose you want to expose to Objective-C a Cangjie struct `Vector` defined as follows:

```
package cj

import interoplib.objc.*

public struct Vector {
    private let _x: Int32
    private let _y: Int32

    public prop x: Int32 { get() { _x } }
    public prop y: Int32 { get() { _y } }

    public init(x: Int32, y: Int32) {
        _x = x
        _y = y
    }
```

```
    public func add(v: Vector): Vector {
        Vector(x + v.x, y + v.y)
    }
}
```

First, you need to import the `interoplib.objc` package into the source code file where `Vector` is defined:

```
import interoplib.objc.*
```

When compiling `Vector.cj`, you would invoke the `cjc` compiler with two additional options: `--experimental --enable-interop-cjmapping=ObjC`. The compiler would then generate two extra files, `Vector.h` and `Vector.m`, similar to the following:

```
// Vector.h
#import <Foundation/Foundation.h>
#import <stddef.h>

__attribute__((objc_subclassing_restricted))
@interface Vector : NSObject

/* glue code */

- (id)init:(int32_t)x :(int32_t)y;

@property (readonly, getter=x) int32_t x;
- (int32_t)x;
@property (readonly, getter=y) int32_t y;
- (int32_t)y;

- (Vector*)add:(Vector*)v;

/* glue code */

@end
```

```
// Vector.m

/* glue code */

@implementation Vector

/* glue code */

- (id)init:(int32_t)x:(int32_t)y {
    /* Glue code constructing an instance of Cangjie Vector(x,y) and associating
```

```
     * it with 'self'.
     */
}
- (int32_t)x {
    /* Glue code retrieving the value of the 'x' property of the
     * assocated instance of Cangjie Vector.
     */
}
- (int32_t)y {
    /* Glue code retrieving the value of the 'y' property of the
     * assocated instance of Cangjie Vector.
     */
}
- (Vector*)add:(Vector*)v {
    /* Glue code invoking the 'add' mmber function of the Cangjie Vector
     * instance associated with 'self', passing over the Cangjie Vector
     * instance associated with 'v', and wrapping the result in a new
     * instance of the Objective-C Vector class.
     */
}

/* more glue code */

@end
```

Now you can add `Vector.h` and `Vector.m` to your XCode project and use the class `Vector` in your Objective-C code as if it was a normal Objective-C class: define variables of the type `Vector*`, create instances, pass them around, including to Cangjie code via `[Vector add]`, etc.

IMPORTANT: The current version of the Cangjie SDK for iOS also requires downloading and integrating into the XCode project the `Cangjie.h` header file from the Cangjie open source repository:

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

Limiting the Exposure

The option `--enable-interop-cjmapping` instructs the `cjc` compiler to generate mirror types for *all* `public` Cangjie types that it compiles, and expose *all* `public` member functions and constructors of those types. Oftentimes, however, such total exposure is undesirable, as what you want to use in Objective-C may be just a small percentage of the functionality provided by a Cangjie library or framework.

You may precisely control the exposure of Cangjie types and their members by passing the pathname of a dedicated configuration file to `cjc` via the option

`--import-interop-cj-package-config-path`. That file is a plain text file the contents of which must conform to the TOML syntax.

In the `[default]` section of that file, you can specify the default API exposure strategy for all Cangjie packages, and then modify it for particular packages in the respective `[[package]]` entries. For example:

```
[default]
APIStrategy="None"       # Expose nothing by default

[[packages]]
name="com.example.pkg1"  # From this specific package,
APIStrategy="Full"       # expose everything
#   .  .  .
```

Furthermore, for each package you can use either the `included_apis` or `excluded_apis` property to respectively expose or hide particular types and/or members:

```
#   .  .  .
[[packages]]
name="com.example.pkg2"          # From this specific package,
included_apis = [ "Vector",      # Only the type 'Vector'
                  "Vector.add"   # and its member function 'add'
                ]                # are exposed

[[packages]]
name="com.example.pkg3"          # From this specific package,
APIStrategy="Full"               # everything
excluded_apis = [ "TopSecret",   # but the type 'TopSecret' and
                  "Auth.getPwd"  # member function 'Auth.getPwd'
                ]                # are exposed
```

Refer to Cangjie Mirror Generation Reference for more details.

Generic Cangjie Types Mirroring

Many important Cangjie types are parameterized. Unfortunately, certain fundamental differences between Cangjie and Objective-C generics preclude mirroring of the former into the latter. As a workaround, the Cangjie SDK supports *monomorphization* of mirror types: it generates a separate *non-generic* Objective-C mirror type for each supplied set of valid type arguments for the given parameterized Cangjie type.

NOTE: The current version only supports mirroring of generic types instantiated with primitive types used as type arguments.

For instance, if you have a generic Cangjie class `p.Pair<T,U>`:

```
package p
public class Pair<T,U> { . . . }
```

and want to manipulate instances of `Pair<Int, Bool>` in your Objective-C code, you would add a `generic_object_configuration` property to the `[[package]]` entry for `p` in the `cjc` mirror generation configuration file, with the following content:

```
#   .   .   .
[[package]]
name = "p"
generic_object_configuration = [
    { name = "Pair", type_arguments = [ "Int, Bool" ] },
#       .   .   .
```

`cjc` would then generate a non-generic Objective-C class:

```
@interface PairIntBool
   .   .   .
@end
```

representing that particular *instantiation* of the type `Pair<T,U>`.

Refer to Generics Instantiations for more details.

Cangjie to Objective-C Mapping

The current version of the `cjc` compiler uses Cangjie → Objective-C mapping described in this chapter. See Cangjie Mirror Generation Reference for applicable `cjc` command-line options.

General Considerations

The Cangjie type system is different from that of the Objective-C language. Certain Cangjie types and their features do not have direct or even reasonably close equivalents in Objective-C, so they have limited support in the current version, and some are not supported at all.

Consequently, if the type of a public member, function/constructor parameter, or function return type is not supported, the respective member, function or constructor cannot be mirrored. The current compiler does not report such usages as errors, the respective entity is simply not mirrored.

Mirror types should be compiled into a separate dynamic library with ARC support disabled (`-fno-objc-arc` Objective-C compiler option). For details, see the notice in the [Classes and Interfaces(#cangjie-classes-and-interfaces) section.

Names

The original names of Cangjie packages, functions, types and type members are currently preserved, which means that some of them may clash with Objective-C keywords, such as "`int`", or contain characters that are not permitted in Objective-C names.

Boolean and Numeric Types

Cangjie Boolean and numeric types that have equivalents among Objective-C types are mirrored to those equivalent; those that don't (`Float16`) are not supported:

Cangjie type	Objective-C type
`Bool`	`BOOL`
`Int8`	`int8_t`
`Int16`	`int16_t`
`Int32`	`int32_t`
`Int64`	`int64_t`
`Int`	`int64_t`
`IntNative`	`ssize_t`
`UInt8`	`uint8_t`
`UInt16`	`uint16_t`
`UInt32`	`uint32_t`
`UInt64`	`uint64_t`
`UInt`	`uint64_t`
`UIntNative`	`size_t`
`Float16`	Not supported
`Float32`	`float`
`Float64`	`double`

See General Considerations for information about the handling of unsupported types.

Rune

The type `Rune` is not supported.

See General Considerations for information about the handling of unsupported types.

Special Types

The type `Unit` is only supported as a function return type, mapped to Objective-C `void`.

The type `Nothing` cannot be supported.

The type `Any` is not supported yet.

See General Considerations for information about the handling of unsupported types.

Tuple

Tuple types are not supported yet.

See General Considerations for information about the handling of unsupported types.

Struct Types

Public Cangjie struct type definitions are mirrored into Objective-C class definitions attributed with `objc_subclassing_restricted`. Those definitions are generated automatically by the `cjc` compiler and contain glue code that transfers control and data between Objective-C and Cangjie. They should not be modified manually.

```
package cj

import interoplib.objc.*

public struct Vector {
    let x: Int32
    let y: Int32

    public init(x: Int32, y: Int32) {
        this.x = x
        this.y = y
    }

    public func add(v: Vector): Vector {
        Vector(x + v.x, y + v.y)
    }
}
```

```
// Vector.h
#import <Foundation/Foundation.h>
#import <stddef.h>
__attribute__((objc_subclassing_restricted))
@interface Vector : NSObject
// Auxiliary glue code methods
- (id)init:(int32_t)x :(int32_t)y;
- (Vector*)add:(Vector*)v;
```

```
// More auxiliary glue code methods
@end

// Vector.m

// Glue code

@implementation Vector

// Glue code

- (id)init:(int32_t)x :(int32_t)y {
    // Glue code creating an instance of Cangjie Vector(x, y) and
    // associating it with 'self'.
}
- (Vector*)add:(Vector*)v {
    // Glue code retireving instances of Cangjie Vector associated
    // with 'self' and 'v', invoking the add() member function
    // of Cangjie-self with Cangjie-v passed a parameter, and then
    // creating a new instance of Vector and associating it with the
    // result of the add() call.
}

// Glue code

@end
```

Only public struct members and common constructors are mirrored.

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. Instance member functions are mirrored into instance methods (prefixed with a minus sign `-`), `static` member functions are mirrored into class methods (prefixed with a plus sign `+`).

Member properties of supported types are mirrored into `@property` declarations with the same name and with type that mirrors the Cangjie property type, attributed with `readonly`. The getter is mirrored into a parameterless method with the same name as the property and return type that is the mirror of the Cangjie property type. **NOTICE:** Setters are currently *not* mirrored, so mirrors of `mut` member properties are also `readonly`.

Member variables of supported types are mirrored into `@property` declarations with the same name and with type that mirrors the Cangjie variable type, attributed with `readonly`. A getter is synthesized to enable access. **NOTICE:** Setters are currently *not* synthesized for `var` member variables, so mirrors of the latter are also `readonly`.

Constructors are mirrored into constructors with the respective mirror types substituted for parameter types. *This includes the default constructor that might have been implicitly declared.*

The current version imposes a number of severe limitations on the struct types that can be mirrored, to the extent that it may be fair to say that a struct type needs to be designed specifically for exposing it to Cangjie:

- Function overloading is not supported: mirrors of overloaded member functions are generated with the same name, which results in a name clash.
- Both `let` and `var` member variables are mirrored into `readonly` properties in the current version, so the latter may not be altered from Objective-C code. This limitation will be removed in a future version. In the meantime, you may add setter functions as a workaround.
- Mirroring of `mut` instance member functions is a work in progress. They are mirrored, but incorrect glue code is generated in some circumstances.
- Mirrored structs may implement interfaces other than `Any`, but the "implements" relationship is *not* propagated to Objecttive-C: th generated `@interface` directive does not bear the names of mirrors of implemented interfaces in angle brackets `<>` after the class name.
- Mirroring of member operator functions is not supported. They are skipped during mirror generation.
- Mirroring of generic structs is supported through monomorphization. See Generics for details.
- Struct member functions and constructors with unsupported parameter types, as well as member functions with unsupported return types, are not mirrored.
- A mirrored struct may have direct and/or interface extensions, but they are ignored during mirror generation.
- Memeber functions may not have any type parameters.

Classes and Interfaces

Cangjie class and interface definitions are mirrored respectively into Objective-C class and protocol definitions. Those definitions contain automatically generated glue code and should not be altered in any way.

```
package cj

import interoplib.objc.*

public interface Valuable {
    public func value(): Int
```

```
}

public open class Singleton <: Valuable {
    private let _v: Int
    public init (v: Int) { _v = v }
    public func value(): Int { _v }
}

public class Zero <: Singleton {
    public init() { super(0) }
}
```

```
// Valuable.h
@protocol Valuable
- (int64_t)value;
@end

// Singleton.h
// Glue code
@interface Singleton : NSObject
// Glue code
-(id)init:(int64_t)v;
- (int64_t)value;
// Glue code
@end

// Singleton.m
// Glue code
@implementation Singleton
// Glue code
-(id)init:(int64_t)v {
    if (self = [super init]) {
        // Glue code creating an instance of Cangjie Singleton(v)
        // and associating it with `self`.
    }
    return self;
}
- (int64_t)value {
    // Glue code calling the value() member function of the Cangjie
    // Singleton instance associated with 'self' and returning the result
}
// Glue code
@end

// Zero.h
#import "Singleton.h"
__attribute__((objc_subclassing_restricted))
@interface Zero : Singleton
```

```
// Glue code
- (id)init;
// Glue code
@end

// Zero.m
// Glue code
@implementation Zero
- (id)init {
    if (self = [super init]) {
        // Glue code creating an instance of Cangjie Zero()
        // and associating it with `self`.
    }
    return self;
}
@end
```

Identifiers that serve as names of mirrored types and their members are currently preserved even if they clash with Objective-C keywords such as `int`.

Mirrors of non-`open` classes are attributed with `objc_subclassing_restricted`.

`public` members and constructors are mirrored. In addition, `protected open` instance member functions of `open` classes are also mirrored, so that they could be overridden in subclasses. No other class or interface members are mirrored.

> **CAUTION:** As there are neither packages nor namespaces of any other kind in Objective-C, the mirrors of such `protected open` member functions are accessible from anywhere in Objective-C code.

Member variables are not mirrored in the current version and no means for accessing them from Objective-C is provided.

Member properties are not mirrored in the current version.

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. `static` member functions are mirrored into class methods (prefixed with "`+`"), instance member functions – into instance methods (prefixed with "`-`").

Common constructors are mirrored into constructors with the respective mirror types substituted for parameter types. *This includes the default constructor that might have been implicitly declared.*

IMPORTANT: From the outside, types that mirror Canglie classes and interfaces are normal Objective-C classes and protocols in all aspects: they can be inherited/implemented, their methods can be overloaded and overridden with conventional Objective-C methods, types of their instances can be queried, and so on. The differences between the languages, however, pose a few potential problems that require extra care when using such types in Objective-C code:

1. There is no attribute that would make Objective-C methods non-overrideable, so methods that mirror non-`open` member functions of `open` classes *can*, but obviously should not, be overridden.

2. The current implementation does not support overriding of mirrors of `static` Cangjie methods of `open` classes. They are mirrored into Objective-C class methods (those prefixed with a plus sign `+`), and should never be overridden in subclasses. There is no way to enforce this restriction at compile-time.

3. Constructors are not inherited in Cangjie, whereas init methods in Objective-C are inherited just like other methods, which may lead to undesirable effects. For instance, consider the following Cangjie classes:

   ```
   public class A {
       public init() {...}
   }
   public class B <: A {
       private init() {...}
       public init(x: Int) {...}
   }
   ```

   In Cangjie, the class `B` does not expose its parameterless common constructor. However, the mirror of `B` will inherit the parameterless `init` method from the mirror of `A`, so the expression `[[B alloc] init]` will compile and run, but the `init` method inherited from `A` will create and associate with `self` an instance of the Cangjie class `A`, not `B`.

NOTICE: Mirrors of `open` Cangjie classes rely on low-level Objective-C features that are not compatible with Automatic Reference Counting (ARC). That is why all mirror types should generally be compiled into a separate dynamic library with ARC support disabled (`-fno-objc-arc` Objective-C compiler option).

The current version imposes a number of severe limitations on the class and interface types that can be mirrored, to the extent that it may be fair to say that such types need to be designed specifically for exposing them to Cangjie:

- Abstract classes are not mirrored and mirrors of concrete classes that implement abstract classes won't compile.
- Function overloading is not supported: mirrors of overloaded member functions are generated with the same name, which results in a name clash.
- Member variables are not mirrored and no means for accessing them is provided. This limitation will be removed in a future version. In the meantime, you may add getter/setter functions to Cangjie classes manually as a workaround.
- Mirroring of classes that implement interfaces other than `Any` is not currently supported in the sense that explicit interface implementation information gets lost during mirror generation.

- Mirroring of generic classes is supported via monomorphization. See Generics for more information.
- Mirroring of classes/interfaces that contain public member properties, member operator functions, or primary constructors is not supported. Such entities are simply not mirrored without a warning.
- For member functions and constructors, the only universally supported parameter types are `Bool` and numeric types. For member functions, the only universally supported return value types are `Bool`, numeric types and `Unit`. Specifically for member functions and constructors of non-`open` classes, structs, enums, and non-`open` classes are also supportted as parameter and return value types, provided that they are defined in the same package. Member functions and constructors with unsupported parameter types, as well as member functions with unsupported return types, are not mirrored.
- Inheritance relationships between interfaces are not preserved during mirror generation.
- Interface implementation information is not propagated to Objective-C during mirror generation (notice how in the above example the `@interface` directive in `Singleton.h` does *not* end with `<Valuable>`).
- A mirrored class may have direct and/or interface extensions, but they are ignored during mirror generation.

Enums

A Cangjie enum is mirrored into an Objective-C class attributed with `objc_subclassing_restricted`, with `static` methods matching the enum constructors (factory methods) and without a `public` constructor.

```
public enum TimeUnit {
    | Year(Int)
    | Month(Int)
    | Year
    | Month
}
```

```
__attribute__((objc_subclassing_restricted))
@interface TimeUnit : NSObject
// Glue code
+ (TimeUnit*)Year:(int64_t)p1;
+ (TimeUnit*)Month:(int64_t)p1;
+ (TimeUnit*)Year;
+ (TimeUnit*)Month;
// Glue code
@end
```

Enum constructors are mirrored into `static` factory methods with the same names and matching types and numbers of parameters; names of parameters are synthesized (`p1`, `p2`, ...).

Only the `public` enum members are mirrored.

Member functions are mirrored into methods with the respective mirror types substituted for parameter types and return value type. Mirrors of member functions returning `Unit` are mirrored into `void` methods. `static` member functions are mirrored into class methods (prefixed with "`+`"), instance member functions – into instance methods (prefixed with "`-`").

Member properties of supported types are mirrored into `@property` declarations with the same name and with type that mirrors the Cangjie property type, attributed with `readonly`. The getter is mirrored into a parameterless method with the same name as the property and return type that is the mirror of the Cangjie property type. **NOTICE:** Setters are currently *not* mirrored, so mirrors of `mut` member properties are also `readonly`.

The current version imposes a number of limitations on the enum that can be mirrored:

- Function overloading is not supported: mirrors of overloaded member functions are generated with the same name, which results in a name clash.
- Mirrored enums may implement interfaces other than `Any`, but the "implements" relationship is *not* propagated to Objecttive-C: th generated `@interface` directive does not bear the names of mirrors of implemented interfaces in angle brackets `<>` after the class name.
- Mirroring of generic enums is *not* supported yet; will be supported through monomorphization. See Generics for details.
- For member functions and constructors, the only supported parameter types are `Bool` and numeric types. For member functions, the only universally supported return value types are `Bool`, numeric types and `Unit`. Member functions and constructors with unsupported parameter types, as well as member functions with unsupported return types, are not mirrored.
- Mirroring of enums that contain member operator functions is not supported. An attempt to mirror an enum containing such an entity currently results in the generation of invalid Objectove-C code.
- Enum member functions and constructors with unsupported parameter types, as well as member functions with unsupported return types, are not mirrored and no warning is issued.
- A mirrored enum may have direct and/or interface extensions, but they are ignored during mirror generation.

Recursively defined enums are supported:

```
public enum Peano {
    | Z
    | S(Peano)

    public func toInt(): Int {
        match (this) {
                    case Z => 0
            case S(x) => 1 + x.toInt()
        }
    }
}
```

Generics

Objective-C and Cangjie generics are fundamentally different, so the latter cannot be mirrored into the former. However, *specific instantiations* of parameterized Cangjie types, such as `G<Int64>`, are concrete types, and therefore can be mirrored into non-generic Objective-C types. Such types are also called *monomorphized generics*.

NOTE: The current version only supports mirroring of generic types instantiated with primitive types used as type arguments. Mirroring of member functions that have their own type arguments is not supported either.

Use the respective configuration file of the `cjc` compiler to specify which instantiations of which types to mirror. Refer to Generics Instantiations for details.

Example:

When compiling the following Cangjie class definition:

```
public class G<T> {
    private let _t: T
    public init(t: T) { _t = t }
    public func get(): T { _t }
}
```

with the following lines in the respective section of the configuration file:

```
    . . .
    generic_object_configuration  = [
        { name = "G", type_arguments = ["Bool", "Int"] },
    ]
    . . .
```

the compiler will generate Objective-C mirror classes for `G<Bool>` and `G<Int>`, named respectively `GBool` and `GInt`.

Refer to the Cangjie Mirror Generation Reference section for complete instructions.

`null` Handling

As Cangjie has no null type, passing `nil` as a parameter to a mirrored Cangjie method results in a run-time exception in glue code:

```
public class Node {
    private let next: ?Node
    public init() {
        next = None
    }
    public init(next: Node) {      /* Pure Cangjie */
        this.next = Some(next)
    }
}
```

```
__attribute__((objc_subclassing_restricted))
@interface Node
/* glue code */
- (id)init;
- (id)init:(Node *)next;
/* glue code */
@end
   .  .  .
     Node *list = [[Node alloc] init:nil];    /* NullPointerException */
```

Conversely, there is currently no way to return `nil` from a `public` member function of an Objective-C-compatible type.

NOTICE: The current version supports automatic `Option<T>` (un)wrapping with `nil` mapped to `None`, but only for Objective-C pointers to classes and protocols mirrored to Cangjie, not the other way round. See `nil` Handling for details.

Cangjie Mirror Generation Reference

There is no standalone mirror generator facilitating the exposure of Cangjie types to Java and Objective-C code in the Cangjie SDK. The `cjc` compiler itself is capable of generating Java/Objective-C mirror type definitions for Cangjie types right when it compiles the Cangjie source code of those types.

Command-line Syntax

The following additional `cjc` options enable and control the generation of mirrors for Cangjie types during compilation:

`--experimental` *(mandatory)*

This option must be specified as of the current version, since the whole interop feature is still in development, but will not be necessary in the future.

`--enable-interop-cjmapping=Java` *or*
`--enable-interop-cjmapping=ObjC` *(mandatory)*

Enables the generation of mirror type definitions for Cangjie types respectively in the form of Java or Objective-C source code files.

`--output-interop-cjmapping-dir` ***pathname*** *(optional)*

The ***pathname*** must point to a directory into which the `cjc` compiler shall place its output Java or Objective-C source code files containing mirror type definitions. If ***pathname*** does not point to an existing file system location, `cjc` attempts to create a directory there. If ***pathname*** points to something other than a directory, `cjc` terminates with an error message.

The default value of ***pathname*** is either `./java-gen` or `./objc-gen`, depending on whether `--enable-interop-cjmapping` is set respectively to "`Java`" or "`ObjC`".

`--import-interop-cj-package-config-path` ***pathname*** *(optional)*

The ***pathname*** must point to a configuration file that defines which Cangjie types are exposed to Java or Objective-C as mirror types. See Cangjie Mirror Generation Configuration for details.

Cangjie Mirror Generation Configuration

A Cangjie mirror generation configuration file is a plain text file in the TOML syntax. It specifies:

- Non-generic Cangjie types to mirror
- Specific instances of generic Cangjie types to mirror

Defaults

The `[default]` table defines the default settings for packages for which either no package-specific settings are provided at all, or those settings don't override the defaults.

Properties:

`APIStrategy` *(optional)*

A string defining whether all public entities shall be mirrored by default or not. Valid values:

- `"Full"` - all public entities shall be mirrored by default
- `"None"` - no public entities shall be mirrored by default

`GenericTypeStrategy` *(optional)*

A string defining whether generic public entities shall be mirrored by default or not. Valid values:

- `"Partial"` - instantiations explicitly listed in `generic_object_configuration` shall be mirrored
- `"None"` - no generic public entities shall be mirrored by default

Packages

Each entry in the `[[packages]]` array specifies the name of a separate source Cangjie package, and, optionally:

- A filter that defines which types from that package to include in, or exclude from, the set of mirrors
- API mirroring strategy (if other than the default)
- Generic type mirroring strategy (if other than the default)
- Specific generic type instantiations to mirror

Properties:

`name` *(mandatory)*

A string containing the name of the *source* Cangjie package to which the settings in this `[[packages]]` array entry apply.

Example:

```
name = "com.example.VectorMath"
```

`APIStrategy` *(optional)*

A string defining whether all public entities of the given package shall be mirrored by default or not. Valid values:

- `"Full"` - all public entities shall be mirrored by default
- `"None"` - no public entities shall be mirrored by default

If this property is absent, the default value is used.

`included_apis` *(optional)*

An array of strings each containing the name of a public entity that needs to be mirrored. If the array contains the qualified name of a type member, the type itself is also mirrored. Either `included_apis` or `excluded_apis` may be present in a given `[[packages]]` array entry, but not both.

Example:

```
included_apis = [
   "Vector.product", // Vector also exposed
   "HiddenV"
]
```

`excluded_apis` *(optional)*

An array of strings each containing the name of a public Cangjie entity defined in the given package that has to be *not* mirrored. Either `included_apis` or `excluded_apis` may be present in a given `[[packages]]` array entry, but not both.

Example:

```
excluded_apis = [
   "Vector.product", // Even if Vector is exposed, product is not
   "HiddenV"
]
```

`GenericTypeStrategy` *(optional)*

A string defining whether generic public entities defined in this package shall be mirrored by default or not. Valid values:

- `"Partial"` - instantiations explicitly listed in `generic_object_configuration` shall be mirrored
- `"None"` - no generic public entities shall be mirrored by default

If this property is absent, the default value is used.

`generic_object_configuration` *(optional)*

A list of tables defining the specific instantiations of generic entities to be mirrored. See Generics Instantiations for details.

Generics Instantiations The `generic_object_configuration` property of a `[[packages]]` array entry defines which specific instantiations of generic Cangjie entities will be mirrored.

The value of the property is a (possibly empty) array of tables. Each entry of the array has the following two mandatory properties:

`name`

A string containing the name of a public generic Cangjie type or global function defined in the current package.

`type_arguments`

An array of strings, each containing valid type argument(s) for the generic type or global function the name of which is specified in the `name` property.

> Simply put, "valid" means that the type/function can be instantiated with the given string between the angle brackets `< >`.
>
> Consider the following Cangjie class:
>
> ```
> public class G<T> {
>     public func f(t: T): Unit {}
> ```

```
    public func f(b: Bool): Unit {}
}
```

`G<Bool>` cannot be instantiated, so

```
generic_object_configuration  = [
    { name = "G", type_arguments = ["Bool"] }
]
```

would trigger a compiler error.

NOTE: The current version only supports mirroring of generic types instantiated with primitive types used as type arguments.

Example:

Given the following Cangjie definitions

```
public class G<T> { . . . }
public func f<T>(): Unit { . . . }
public struct S<T,U> { . . . }
```

the property

```
generic_object_configuration  = [
    { name = "G", type_arguments = ["Int32"] },
    { name = "f", type_arguments = ["Float32", "Float64"] }
    { name = "S", type_arguments = ["Int, Bool"] }
]
```

instructs the `cjc` compiler to generate the following generics instantiations and mirrors for them:

```
G<Int32>
f<Float32>
f<Float64>
S<Int, Bool>
```

naming the mirrors `GInt32`, `fFloat32`, `fFloat64`, and `SIntBool` respectively.

Using Objective-C in Cangjie

In the first interoperation scenario, described in Using Cangjie in Objective-C, user-defined Cangjie types are mapped to Objective-C types that behave just like any conventional Objective-C classes and protocols: classes can be extended, protocols implemented, instances created and passed around freely, and so on.

The Cangjie SDK for iOS also supports similar features for the reverse scenario, in which Objective-C classes and protocols are mapped to Cangjie classes and interfaces and can be extended, implemented, instantiated. However, the exact set of features and limitations is different, as well as tooling: mirror type and

function declarations for Objective-C entities are generated by a standalone tool, not the `cjc` compiler.

The process of enabling this second scenario is therefore more involved. Here are the steps you need to take:

1. Design the interoperability layer in terms of Objective-C classes and methods.

   developer → `interop layer design` (Objective-C *pseudo-code*)

2. Generate mirror declarations for any Objective-C classes and protocols used in the design of the interoperability layer.

   `.h` → mirror generator → `.cj` (mirrors)

3. Write the classes constituting the interoperability layer *in Cangjie*. Use the mirrored Objective-C types as needed: create instances, call member functions, etc.

   `interop layer design` + `.cj` (mirrors) → developer → `.cj` (interop layer)

4. Compile the interoperability layer classes (*interop classes* for short) and mirror types together with `cjc`. `cjc` will generate:

   - the necessary glue code for all uses of the mirror types.
   - the actual Objective-C source for the Objective-C part of the interoperability layer (so called Objective-C *wrappers* for interop classes)

   `.cj` (mirrors + interop layer) → `cjc` → `.dylib` + `.h`/`.m` (interop layer)

5. Add to your iOS project:

   - `.h` and `.m` files generated by `cjc`
   - `.dylib` file generated by `cjc`
   - runtime libraries (`.dylib`) from the Cangjie SDK

   iOS project + `.h`, `.m`, `.dylib` → iOS toolchain → iOS application

6. Now you may start using the interop class wrappers in the Objective-C code of your application, effectively calling Cangjie code from Objective-C.

IMPORTANT: The current version of the Cangjie SDK for iOS also requires downloading and integrating into the XCode project the `Cangjie.h` header file from the Cangjie open source repository:

https://gitcode.com/Cangjie/cangjie_runtime/blob/dev/runtime/src/Cangjie.h

Initial Interop Class Creation Workflow

Setup

1. Install the `llvm@16` Homebrew formula:

   ```
   brew install llvm@16
   ```

2. Add the `llvm@16/lib/` subdirectory to the `DYLD_LIBRARY_PATH` environment variable:

   ```
   export DYLD_LIBRARY_PATH=/opt/homebrew/opt/llvm@16/lib:$DYLD_LIBRARY_PATH
   ```

3. Run the `envsetup.sh` script from the Cangjie SDK.

4. Verify the installation by running the following command in the terminal:

   ```
   ObjCInteropGen
   ```

   If mirror generator usage instructions start to appear, you now have everything you need to begin using the iOS interoperability features of the Cangjie SDK.

Step 1: Design the Interoperability Layer

On this step, you design the API of one or more interop classes *from the Objective-C code perspective*, i.e. you need to decide, for each interop class:

- What Objective-C class it will extend, e.g. `NSObject`;
- Which Objective-C protocols it will implement, if any; and
- What `public`/`protected` methods and/or constructors it will have. You only need to know the types of parameters and, for methods, their names and return value types; the implementations you will write in Cangjie.

See also Features and Limitations of Interop Classes.

For the Objective-C part of the application, the interoperability layer is indistinguishable from a set of regular Objective-C classes. So you can design it in the form of Objective-C pseudo-code. Write down the specifications and `@interface` definitions of one or more Objective-C classes. **Note:** You do not need to write the respective `@implementation` parts as you would re-write them in Cangjie at a later step anyway.

Supported parameter types: any mapped Objective-C types.

Supported return types: any mapped Objective-C type or `void`.

Supported inheritance: interop classes may only extend mirrored Objective-C classes (*not* other interop classes!), and may only implement interfaces that mirror Objective-C protocols.

Limitations:

- Variable arity (varargs) methods and constructors are not supported.

- Neither an interop class nor its individual non-private member functions may be generic.
- Generic Objective-C types are erased before mirror generation; any type variables are replaced with the respective upper bounds.

End-to-end Example:

For the sake of simplicity, suppose that in your iOS application there is already a class `M` with a single parameterless `void` instance method `foo()`:

```
// M.h
#import <Foundation/Foundation.h>

@interface M : NSObject
- (void)foo;
@end
```

```
// M.m
#import "M.h"

@implementation M
- (void) foo {
    printf("Hello from ObjC M.foo()\n");
}
@end
```

and you want to override that method `foo()` with a Cangjie implementation in a subclass of `M` called `A`.

Your interop class design would then look like this:

```
#import "M.h"
@interface A : M
- (void)foo;
@end
```

Step 2: Generate Mirror Type Declarations

Now, you need mirror type declarations for all Objective-C types on which your interop classes depend: their supertypes, types of member properties and local variables, types of method parameters/return values, and, if any of the foregoing are array types, their element types.

Before you proceed further, build your iOS application normally without changing anything in it, to ensure that the mirror generator takes a complete and consistent set of Objective-C headers as input.

Then write an appropriate mirror generator configuration file and run the mirror generator:

```
ObjCInteropGen --mode=normal <config-file>
```

where `<config-file>` is the pathname of the configuration file.

End-to-end Example (continued):

The only immediate dependency of your class `A` is its superclass `M`, so the configuration file is pretty simple:

```
# A.toml
# Place the mirror of M and any dependencies it may have in the 'cjworld' package:
[[packages]]
filters = { include = ["M", "NS.+"] }
package-name = "cjworld"

# Write the output files with mirror type definitions to the current directory:
[output-roots.default]
path = "."

# Specify the pathname of the input header:
[sources.all]
paths = ["M.h"]

[sources-mixins.default]
sources = [".*"]
arguments-append = [
    # Uncomment the following line if you get "unknown type name" errors
    # "-DTARGET_OS_IPHONE=1",

    # Edit the pathnames below to match the locations of Objective-C headers on your system:
    "-F", "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/System/Library/Frameworks",
    "-isystem", "/Library/Developer/CommandLineTools/usr/lib/clang/17/include",
    "-isystem", "/Library/Developer/CommandLineTools/SDKs/MacOSX.sdk/usr/include"
]
```

Mirror generator command line:

```
ObjCInteropGen --mode=normal A.toml
```

The above command will generate files `cjworld/M.cj` with a mirror type declaration for the class `M` and a bunch of `cjworld/NS*.cj` files with mirror type declarations for the Foundation framework classes and protocols on which `M` depends.

Common Misconfiguration Issues The mirror generator is unable to find the standard header files, such as `stdarg.h` or `stdbool.h`.

Example error messsage:

```
..../CoreFoundation.h:19:10: error: 'stdarg.h' file not found
```

This usially means that the pathnames in `arguments-append` array of the `[sources-mixins]` table are not correct. Double check that they match the locations of the header files on your system.

The mirror generator reports "Unknown type name 'NSUInteger'" errors or similar.

Example error messsage:

```
.../NSObjCRuntime.h:626:74: error: unknown type name 'NSUInteger'
```

On some systems, you need to pass an additional argument to Clang, either "`-DTARGET_OS_IPHONE=1`" or "`-DTARGET_OS_OSX=1`".

Add it to the `arguments-append` array of the `[sources-mixins]` table. In the above example, it is present but commented out:

```
   .  .  .
arguments-append = [
    # Uncomment the following line if you get "unknown type name" errors
    # "-DTARGET_OS_IPHONE=1",
   .  .  .
```

Step 3: Write the Interop Classes

For each Objective-C class skeleton that you included in your interop layer design on Step 1, write a matching Cangjie class as follows:

- Use the appropriate package and class names (the Objective-C wrapper class that `cjc` will generate for you will have the same fully qualified name).
- Import `interoplib.objc.*`.
- Import the mirror types, if you've generated any on Step 2. Do not import all generated dependencies, only the types you actually need.
- Annotate the interop class with `@ObjCImpl`.
- Make the interop class inherit the respective `open` mirror class, or `NSObject` if it has no explicit supertype in your design.
- Annotate `public` class members and constructors with `@ForeignName["`*`foreign-name`*`"]`, where *`foreign-name`* is the desired Objective-C name for that member or constructor, according to the following rules:
    1. *Do not* annotate member functions that override member functions of mirror supertypes. The *presence* of a `@ForeignName` annotation on such a member function of an interop class triggers a compile-time error.

        The Objective-C names of such overriding functions must match the names of the overridden Objective-C methods.

The latter are propagated *from* Objective-C to Cangjie automatically via `@ForeignName` annotations in mirror type declarations (see Classes and Protocols for details). The `cjc` compiler picks those annotations up.

2. *Do* annotate constructors and non-overriding member functions that have *two or more* parameters. The *absence* of a `@ForeignName` annotation on such a member function or constructor triggers a compile-time error.

   The compiler is unable to morph the original Cangjie name of a member function or constructor with two or more parameters into a valid Objective-C name automatically. Such names essentially consist of the respective number of fragments separated with the colon character `:`.

3. Overloaded member functions and constructors *must* be given distinctive Objective-C names using `@ForeignName`, as there is no method overloading in Objective-C. Exacly one of such overloaded entities can retain its original Cangjie name, provided it has at most one parameter.

4. Annotating the remaining members and constructors with `@ForeignName` is optional. In the absence of a `@ForeignName` annotation, the original Cangjie name of such a member is used as the Objective-C name of the corresponding entity, with colon `:` appended to names of single-parameter methods, whereas the `init`-method matching the sole `public` constructor with one or zero parameters is called respectively `init:` or `init`.

NOTE: The current version of the `cjc` compiler does *not* fully validate *`foreign-name`*. In particular, it does not check that the number of colon `:` separators mathes the number of method or constructor parameters.

- Use the following Objective-C-to-Cangjie type mapping (`T'` is either the matching value type or the respective mirror type):

Objective-C	Cangjie	Remark
`void`	`Unit`	
`BOOL`	`Bool`	
`signed char`	`Int8`	
`short`	`Int16`	
`int`	`Int32`	
`long`	`Int64`	
`long long`	`Int64`	
`unsigned char`	`UInt8`	
`unsigned short`	`UInt16`	
`unsigned int`	`UInt32`	

Objective-C	Cangjie	Remark
`unsigned long`	`UInt64`	
`unsigned long long`	`UInt64`	
`float`	`Float32`	
`double`	`Float64`	
`struct`	`@C struct'`	(*)
`enum` w/base type `T`	`T'`	
`id`	`ObjCId`	(†)

Objective-C	Cangjie	If `T` is...	Remark
`T*`	`CPointer<T'>`	... a primitive or structure type	
`T*`	`CPointer<U'>`	... an enum type and `U` is its base type	
`T*`	`CFunc<T>`	... a pure C function type	
`T*`	`T'`	... a class type	(†)

(*) The Objective-C structure must not contain fields of types that are not `CType`-compatible. See Structs for details.

(†) Use `?<T'>` (`Option<T'>`) types for parameters, return values, and local variables of mirror types and interop classe types that may receive/hold the Objective-C `nil` value.

See Objective-C to Cangjie Mapping for details.

Supported features:

Member functions of interop classes:

- Can override/redefine `open` instance/static member functions of their mirrored superclasses, implement member functions of their mirrored superinterfaces, and be newly introduced member functions.

Constructors and member functions of interop classes:

- Can use any Cangjie language features in the body when working solely with (values of) regular Cangjie types.
- Can use conventional Cangjie syntax to:
  - instantiate objects of interop classes and mirrored Objective-C classes
  - call static and instance methods of interop classes and mirrored Objective-C types (that includes using `super` to call Objective-C superconstructors and superclass methods)
  - access their own member properties and non-`private` member properties of other interop classes and mirrored Objective-C types

Limitations:

- Interop classes may implement interfaces that are mirrored Objective-C protocols, but not regular Cangjie interfaces.
- Interop classes may not be declared as `open` or `abstract` and may not be extended using `extend`.
- Non-`private` member functions and constructors of interop classes:
  - May only use the Objective-C-mapped types listed above as parameter and return value types. (As a consequence, variable-length arguments are not supported as that requires using the Cangjie `Array<T>` type, which has no Objective-C mapping.)
  - May not have named parameters.
  - May not have type parameters, i.e. be generic.
- Non-`private` member properties may only have the Objective-C-mapped types listed above.
- Generic Objective-C types are erased; mirror types have upper bounds in place of the respective type parameters.
- **IMPORTANT:** Instances of mirror types and interop classes (in other words, values of Objective-C reference types) must not escape to global or static Cangjie variables or any data structures that persist between calls.

End-to-end Example (continued):

To continue the above example, the interop class `A` would look like this:

```
package cjworld           // Same package name

import interoplib.objc.*  // Always required

@ObjCImpl
public class A <: M {

    public init() {
        super()
    }

    @ForeignName["foo"]
    override public open func foo(): Unit {
        println("Hello from overridden A.foo()")
    }

}
```

Step 4: Compile the Interop Classes

Command line:

```
cjc --target=arm64-apple-ios-simulator \
    --sysroot=$(xcrun --show-sdk-path --sdk iphonesimulator) \
    --output-type=dylib \
    --int-overflow=wrapping \
    <source-files> \
    -o <target-file> \
    --link-options "-undefined dynamic_lookup"
```

where

`<source-files>` are the source code files of the interop classes and mirror type declarations.

`<target-file>` is the desired name of the output `.dylib` file with compiled Cangjie code for the interop classes, such as `libcjworld.dylib`.

The compiler will also generate Objective-C source files (`.h` and `.m`) for the Objective-C wrappers or the interop classes, placing them in the `./objc-gen/` subdirectory.

The `.dylib` file has to be signed:

```
xcrun codesign --sign - <dylib-file>
```

End-to-end example (continued):

Compile the interop class:

```
cd cjworld
```

```
cjc --target=arm64-apple-ios-simulator \
    --sysroot=$(xcrun --show-sdk-path --sdk iphonesimulator) \
    --output-type=dylib \
    --int-overflow=wrapping \
    *.cj \
    -o libcjworld.dylib \
    --link-options "-undefined dynamic_lookup"
```

The compiler will produce three files: `./libcjworld.dylib`, `./objc-gen/A.h` and `./objc-gen/A.m`.

Sign the generated dynamic library:

```
xcrun codesign --sign - libcjworld.dylib
```

Step 5: Put It All Together

- Create another subdirectory in your project and copy all dynamic libraries from the `$CANGJIE_HOME/runtime/lib/ios_simulator_aarch64_cjnative/`

directory into it.

- Add those all dynamic libraries *and* the `.dylib` file generated on the previous step to your Xcode project as dependencies (BuildPhases - in both "Copy Files" and "Link Binary With Libraries" lists).
- Move the `.h` and `.m` files generated on the previous step to project root.

Then re-build the project.

End-to-end Example (continued):

Create a new subdirectory in the root directory of your project and copy the dynamic libraries constituting the Cangjie Runtime for iOS into it:

```
cd ..
mkdir -p CJRuntimeDylibs
cp $CANGJIE_HOME/runtime/lib/ios_simulator_aarch64_cjnative/*.dylib CJRuntimeDylibs/
```

Add those dynamic libraries *and* `./cjworld/libcjworld.dylib` to your Xcode project as dependencies (BuildPhases - in both "Copy Files" and "Link Binary With Libraries" lists).

Move the `.h` and `.m` files generated by `cjc` to project root:

```
mv cjworld/objc-gen/*.h ./
mv cjworld/objc-gen/*.m ./
```

Rebuild the Xcode project.

Calling Objective-C from Cangjie

Once you have designed, built and integrated the interoperability layer as described in the previous section, you can add code that uses Objective-C types to the member functions of your interop classes. The type mapping is the same:

Add the code invoking a Cangjie function from Objective-C via the respective method of an interop class wrapper and re-build your iOS project again.

End-to-end Example (continued):

Insert an instantiation of `A` and a call of the method `foo()` of that instance into your Objective-C code:

```
        .   .   .
#import "M.h"
#import "A.h"
        .   .   .
    M* a = [[A alloc] init];
    [a foo];
        .   .   .
```

Rebuild the Xcode project and run your app on the iOS simulator.

Subsequent Enhancements

Following the above process should have helped you develop a good understanding of how to add more classes to the interoperability layer and/or enable the use of more Objective-C classes in its Cangjie code. Below please find a brief summary with links to the respective subsections:

Step 1: Add more interop classes to your design and/or enhance the existing ones with new methods and/or properties as you wish.

Step 2: If any additional, not yet mirrored Objective-C reference types will now get involved, build the *current* version of the application to ensure consistency, edit the mirror generator configuration file appropriately, and then (re-)generate all mirror type declarations.

Step 3: Implement the newly added interop classes, if any, and/or change the code of existing ones, using the mirrored Objective-C types as necessary.

Step 4: Compile all interop classes together.

Step 5: Copy the `.h`, `.m` and `.dylib` files that `cjc` has generated on the previous step to their respective locations in the project and re-build it.

Now you can start using the newly added interop classes and/or methods in the Objective-C part of your application.

Features and Limitations of Interop Classes

1. An interop class *must* be a direct subclass of a mirror class.
2. An interop class *may* implement one or more mirror interfaces, but never a conventional Cangjie interface. Conversely, a conventional Cangjie type may not implement or inherit an interface mirroring an Objective-C protocol.
3. An interop class may not be declared as `open` or `abstract` and may not be extended using `extend`.
4. An interop class may introduce new instance fields *of any Cangjie type* and override the member functions of its mirrored superclass.
5. The constructors of an interop class may call superclass constructors using `super()`, but have the same limitation on instance member function calls as the conventional Cangjie constructors, as well as the requirement to initialize all newly introduced member variables.
6. The instance member functions of an interop class may call instance member functions which that interop class has inherited from its mirrored superclass, and/or use `super.` to call the member functions of that superclass that the interop class overrides.

7. The signatures of constructors and member functions of all mirror types and interop classes can only use types that are (a) mirror types or interop classes themselves, or (b) are 100% analogous to Objective-C primitive types. See the Type Mapping table in Step 3 above and the Chapter Objective-C to Cangjie Mapping.

Objective-C to Cangjie Mapping

The current version of the mirror generator does the following Objective-C → Cangjie conversions.

General Considerations

The Objective-C mirror generator relies on Clang for parsing Objective-C sources, invoking it with the `-fobjc-arc` option.

Declarations marked with the `unavailable` attribute are ignored.

Global, file-level function and variable declarations are also ignored.

The order of declarations in the output Cangjie source code is the same as the order of the respective definitions in the input Objective-C source code, with the exception of nested type definitions.

Names

The original Objective-C identifiers are preserved, with the following exceptions:

- Objective-C identifiers that clash with Cangjie keywords, such as `catch`, `false`, or `UInt32`, are enclosed in backticks ``.
- Name conflicts between Objective-C classes and protocols are resolved by adding the suffix `Protocol` to the name of the protocol.
- Name conflicts between instance and static methods are resolved, if possible, by adding either `Instance` or `Static` suffix to the highest (closest to the root) class/protocol where the conflict is encountered. If the conflicting instance/static methods are first introduced in the same class/protocol, the static method is renamed. For example:

```
@interface A
+(void)foo;
@end

@interface B : A
-(void)foo;
+(void)bar;
-(void)bar;
@end
```

will be converted to

```
@ObjCMirror
public open class A <: ObjCId {
    public static func foo()
}

@ObjCMirror
public open class B <: A {
    public open func fooInstance()
    public static func barStatic()
    public open func bar()
}
```

- Conflicts between `init` methods having different names, but the same number and types of parameters are not resolved. See Classes for details.

Type Aliases

C typedef declarations are converted to public Cangjie type aliases.

Primitive Types

Objective-C primitive types map to the respective Cangjie value types. C types with platform-specific sizes map to Cangjie in accordance with their actual sizes on the *host* platform, i.e. the platform on which the mirror generator runs. For example, on macOS it can be the following:

Objective-C	Cangjie
`void`	`Unit`
`BOOL`	`Bool`
`signed char`	`Int8`
`short`	`Int16`
`int`	`Int32`
`long`	`Int64`
`long long`	`Int64`
`unsigned char`	`UInt8`
`unsigned short`	`UInt16`
`unsigned int`	`UInt32`
`unsigned long`	`UInt64`
`unsigned long long`	`UInt64`
`float`	`Float32`
`double`	`Float64`

Structs

A C struct is converted to a public Cangjie struct marked as `@C` if it is `CType`-compatible, that is, contains fields of `CType`-compatible types only. Other scenarios, e.g. the original Objective-C structure containing pointers to objects, are not currently supported.

Nested structures are converted to top-level Cangjie structs.

Incomplete structure declarations are converted to empty structs (without any members).

Fields of Objective-C structures are converted to public member variables of the respective Cangjie structs. The `static` modifier is maintained.

Bit fields are not supported by the Cangjie language, so their widths are ignored and a warning is issued.

The Cangjie language, unlike Objective-C, requires all member variables of structs (even `@C` ones) to be initialized. Therefore, mirrored structures contain explicit zero initializers for all fields. For example:

```
struct A {
    int x;
    double y;
    BOOL z;
    struct A *w;
};
```

is mirrored to

```
@C
public struct A {
    public var x: Int32 = 0
    public var y: Float64 = 0.0
    public var z: Bool = false
    public var w: CPointer<A> = CPointer<A>()
}
```

Enumerations

Named C enumeration declarations are converted to abstract sealed Cangjie classes (effectively, namespaces). Enumerators are converted to public static constants initialized with their values. Their type corresponds to the explicitly specified underlying type for the enumeration, if any, otherwise it is `Int32`.

Unions

The Cangjie language does not support C union types. They are converted to structs with sequential fields and a warning is issued.

Classes and Protocols

Objective-C classes and protocols are mirrored respectively to Cangjie classes and interfaces. All such mirror classes and interfaces explicitly implement the built-in `ObjCId` mirror interface (see Built-in Types).

In methods with a variable number of parameters, `, ...` is ignored.

Full Objective-C method names (selectors) decorated with `:` are not valid Cangjie identifiers. Such names are mangled as follows:

- Each letter that immediately follows a `:` character, if any, is capitalized.
- All `:` characters are removed.

The original selector is specified in the `@ForeignName` attribute on the Cangjie side.

Example:

```
@interface A
- (void)foo;
- (void)foo:(int)i;
- (void)foo:(int)i bar:(int) j;
- (void)foo:(int)i bar:(int) j baz:(int) k;
@end
```

is mirrored to

```
@ObjCMirror
public open class A {
    public open func foo(): Unit
    @ForeignName["foo:"] public open func foo(i: Int32): Unit
    @ForeignName["foo:bar:"] public open func fooBar(i: Int32, j: Int32): Unit
    @ForeignName["foo:bar:baz:"] public open func fooBarBaz(i: Int32, j: Int32, k: Int32): U
}
```

In Cangjie, a property that overrides another property must have the same type. Also, the getter/setter functions of Cangjie properties may not be named arbitrarily. Hence any properties declared in Objective-C classes are only mirrored into Cangjie properties *when possible*. In all other cases, their getter/setter methods are mirrored as ordinary instance member functions. This includes properties declared within `@protocol` directives.

Instance variables are mirrored into instance member variables.

`instancetype` is converted to the name of the current declaration.

Classes

Objective-C `@interface` class declarations are converted to Cangjie classes marked as `@ObjCMirror public open`.

Objective-C `@interface` category and extension declarations are merged with the respective Cangjie class declarations.

Objective-C `@implementation` declarations are ignored.

Forward class declarations (`@class` directives) are mirrored into empty classes (without any members).

Methods identified as `init` methods as described in the Method families section of the Clang documentation on Objective-C ARC are mirrored into Cangjie constructors, with two limitations:

- If two or more `init` methods of a class differ only by name, i.e. have the same number and types of parameters, the mirror generator comments out the respective Cangjie constructor declarations and issues a conflict warning.
- `init` methods are inherited like any other methods, whereas constructors in Cangjie are not inherited. The constructors that mirror the `init` methods of superclasses are therefore unavailable at the point of a mirror class or interop class instantiation.

Other methods of Objective-C classes are mirrored into public open member functions of the respective Cangjie mirror types. Mirrors of class methods (those with the "`+`" prefix) are modified with `static`.

Protocols

Objective-C `@protocol` directives are mirrored into Cangjie interfaces marked as `@ObjCMirror public`. However, the cjc compiler does not support such interfaces yet.

Forward protocol declarations are mirrored into empty interfaces (without any members).

Methods of Objective-C protocols are mirrored into public open member functions of the respective Cangjie interfaces. The `static` modifier is maintained.

The `@optional` directive is ignored.

Pointers

The modifiers `const`, `volatile`, and `restrict` are ignored.

Pointers to C primitives and their type aliases are converted to `CPointer` with the respective type parameters.

A pointer to a C enumeration is converted to a `CPointer` to its underlying type, with the actual enumeration name specified in a comment.

Pointers to Structures

A pointer to a C structure `T` is converted to `CPointer<T'>`, where `T'` is the mirror type for `T`, if the structure is `CType`-compatible. Otherwise, they are converted to `ObjCPointer<T'>` (provisional name), which is a type that should be implemented in the interop library.

Pointers to Functions

Pointers to C functions are converted to either `CFunc` (if the function parameter types and return value are all `CType`-compatible) or to `ObjCFunc<F>` (provisional name) otherwise. The latter is a type implemented in the interop library.

Pointers to Blocks

Pointers to Objective-C blocks are converted to `ObjCBlock<F>` (provisional name), which is a type implemented in the interop library. The type argument `F` of `ObjCBlock<F>` must be the respective Cangjie function type.

Pointers to Class Instances

Pointers to Objective-C class instances are mirrored as follows:

- Pointers to class instances (`SomeClass*`) are mirrored to the respective mirror classes.
- The mirror of the Objective-C type `id` narrowed with a single protocol (for example, `id<NSCopying>`), is the mirror interface for that protocol.
- `id` narrowed with several protocols, e.g. `id<NSCopying, NSSecureCoding>`, is mirrored to a pure `ObjCId` (the interface that all `@ObjCMirror` classes and interfaces respectively implement and inherit), with the actual protocol list specified in a comment.
- If a generic type parameter is used inside a generic template with a narrowing protocol specified, it is converted to a reference to that protocol, with the name of the type parameter specified in a comment.

Generics

Parameterized Objective-C classes are mirrored as regular, non-generic Cangjie class declarations. The original lightweight generics syntax such as "`<T>`" or "`<Foo>`" is retained in comments; type parameter usages are replaced with `ObjCId`.

Example:

```
@interface G<T> : NSObject
- (void)f:(T)t;
@end
```

is mirrored to

```
@ObjCMirror
public open class G/*<T>*/ <: NSObject {
    @ForeignName["f:"] public open func f(t: ?ObjCId /*T*/): Unit
}
```

However. type constraints are lost as of the current version, i.e.

```
@interface G<T: SomeType*> : NSObject
- (void)f:(T)t;
@end
```

is now mirrored exactly as the first sample above.

Built-in Types

The mirror generator assumes that the following user-available Cangjie types are implemented in the interop library. As of the current version, however, they are not yet implemented, and their names may change.

Objective-C	Cangjie (*)	Description
`id`	`ObjCId`	Interface that should be implemented by all `@ObjCMirror` classes and interfaces. Bound to Objective-C `id`.
`SEL`	`SEL`?	Class bound to Objective-C `SEL`.
`Class`	`Class`?	Class bound to Objective-C `Class`.
`Protocol`	`Protocol`?	Class bound to Objective-C `Protocol`.
pointer type	`ObjCPointer<T>`?	Structure bound to Objective-C pointers of two kinds: with arity more than one and when `T` is a not `CType`-compatible structure.
non-C function type	`ObjCFunc<F>`?	Class implementing `CFunc` for functions that are not `CType`-compatible. `F` is a Cangjie function type.
block type	`ObjCBlock<F>`?	Structure implementing an Objective-C block. `F` is a Cangjie function type.
	`__builtin_va_list`	Helper type alias for `CPointer<Unit>`, technically needed in the current generator implementation. May and should be dropped in the future.

(*) Names of Cangjie types are provisional.

Not (Yet) Implemented Features

Objective-C interoperability support in the Cangjie SDK is still in development. Certain features are not implemented yet, others may change in the first production release, and some cannot be (fully) implemented due to principal differences between the two languages.

- The C language features that alter the default size, packing, padding, and/or alignment of data are very ABI-specific. The Cangjie language does not support such low-level features, and bit fields in particular. The mirror generator therefore ignores bit field width specifiers and issues warnings.
- The Cangjie language does not provide support for C union types, so they are mirrored as structs and warnings are issued.
- Structures with fields of types that are not `CType`-compatible are not supported.
- Anonymous C enumeration declarations are ignored. Named ones are mirrored to abstract sealed Cangjie classes. See Enumerations for details.
- In methods with a variable number of parameters, `...` is ignored.
- The `@optional` directive is ignored.
- Properties are mirrored as properties if possible, otherwise their getter and setter methods are mirrored as if they were regular instance methods. See Classes and Protocols for details.
- The modifiers `const`, `volatile`, and `restrict` are ignored.
- `init` methods are mirrored into Cangjie constructors, which imposes two limitations:
  - Two constructors of a Cangjie class may not have the same number and types of parameters, whereas `init` method may differ by name, and
  - Constructors in Cangjie are not inherited.

  See Classes for details.
- Parameterized Objective-C classes are mirrored as regular, non-generic Cangjie class declarations. See Generics for details.
- Mirroring of `@protocol` declarations into Cangjie interfaces is not fully supported yet.
- Pointers to functions with parameter/return types that are not `CType`-compatible are not supported.
- Objective-C blocks are not supported.

- The intrinsic helper functions for converting values of the Objective-C type `NSString` to/from values of the Cangjie type `String` are not implemented yet.

`nil` Handling

Cangjie has no concept of null references and hence no equivalent for the Objective-C `nil` value. If pointers to Objective-C classes and protocols were mirrored to Cangjie classes and interfaces, any such value passed over from Objective-C to Cangjie could lead to a segmentation fault. Conversely, there would be no way to pass a `nil` value from Cangjie to Objective-C either.

The `Option<T>` enum is therefore generally used to represent the values of those Objective-C types, with `None` standing for the `nil` value, and `Some(r)` representing a (non-null) reference value `r`. The `cjc` compiler recognizes `Option<T>` as an Objective-C compatible type if `T` is a mirror type or interop class and wraps/unwraps the values of `T` accordingly.

For instance, the following Objective-C `@interface` directive:

```
@interface MyContainer: NSObject
   .  .  .
- (void)addItem:(MyItem *)item withUuid:(NSString *)uuid;
- (MyItem *)itemWithUuid:(NSString *)uuid;
- (NSString *)uuidForItem:(MyItem *)item;
@property (copy) NSArray<MyItem *> *allItems;
@end
```

will be mirrored to (`@ForeignName` annotations omitted for brevity):

```
@ObjCMirror
open class MyContainer <: NSObject {
       .  .  .
    public open func addItemWithUuid(item: ?MyItem, uuid: ?NSString): Unit
    public open func itemWithUuid(uuid: ?NSString): ?MyItem
    public open func uuidForItem:(item: ?MyItem): ?NSString
    public mut prop allItems ?NSArray/*<MyItem>*/
}
```

The `Option<T>` wrapping ensures that the code won't break if a `nil` value sneaks into the Cangjie world from the Objective-C one, but it does that at the cost of performance and memory footprint. The other disadvantage of this approach is loss of variance. That being said, support for nullability annotations considerably reduces the impact of reference wrapping, at least when it comes to the use of iOS APIs in Cangjie code.

NOTE: This problem does not exist for the low-level C types that get mirrored to the conventional `CPointer<T>` type, as the latter provides explicit null check

functions.

Loss of Variance

One limitation imposed by the `Option<T>` wrapping of Objective-C mirror types and interop classes is that such wrapped types follow the semantics of Cangjie in all other respects. In particular, `Option<T>` is *invariant by its type parameter `T`*: `Option<U>` is not a subtype of `Option<T>` if `U` is a subtype of `T` unless `T` and `U` are the same type. For mirror types that means that any overriding Objective-C method that relies on return type covariance may not be mirrored that way with `Option<T>` wrapping. The return value type of the mirror of such a method has to be propagated from the method that it overrides.

Example:

Suppose an Objective-C class `Foo` is a direct superclass of the class `Bar`:

```
@interface Foo : NSObject
@end

@interface Bar : Foo
@end
```

and the class `C` declares a method `get` that returns an instance of `Foo`:

```
@interface C : NSObject
- (Foo*) get;
@end
```

A subclasss of `C` may then override `get` with a more precise return type, `Bar`:

```
@interface D : C
- (Bar*) get;
@end
```

Without `Option<T>` wrapping, all those classes could be mirrored to:

```
@ObjCMirror
open class Foo <: NSObject {}

@ObjCMirror
open class Bar <: Foo {}

@ObjCMirror
open class C <: NSObject {
    open func get(): Foo
}

@ObjCMirror
open class D <: C {
```

```
    open func get(): Bar       // Return type covariance in action
}
```

but if `get()` can possibly return `nil`, an application crash is inevitable.

`Option<T>` wrapping makes it safe, but the return types of all overriding methods have to be lowered to the return type of the original method:

```
@ObjCMirror
open class Foo <: NSObject {}

@ObjCMirror
open class Bar <: Foo {}

@ObjCMirror
open class C <: NSObject {
    open func get(): Option<Foo>
}

@ObjCMirror
open class D <: C {
    // open func get(): Option<Bar>  // Error, `Option<T>` is not covariant by T
    open func get(): Option<Foo>     // OK, but the return type is lowered
}
```

Nullability annotations alleviate the problem partially.

Nullability Annotations

Nullability keywords were introduced to Objective-C with the release of XCode 6.3 back in the day, for better integration with the new iOS/OS X development language, Swift. The definitions of all iOS APIs were enhanced with nullability annotations to reduce the use of optionals in Swift code utilizing those APIs.

> **Objective-C Nullability Annotations**
>
> The keywords `nullable` and `nonnull` can be used for Objective-C properties, method result types and method parameter types. They mean that the given entity respectively may or may not hold or accept the `nil` value. The (very rare) `null_unspecified` annotation denotes the entities about which it is unknown whether their value may be `nil`.
>
> In addition, any pointer type may be annotated with `__nullable`, `__nonnull` or `__null_unspecified` with the same semantics.
>
> Finally, a property may also ne annotated with `null_resettable`, meaning that its getter will never return `nil`, but passing `nil` to its setter would reset the property to some default value.

For details, see Designating Nullability in Objective-C APIs

All uses of Objective-C reference types annotated as non-nullable are therefore exempt from `Option<T>` wrapping. In other words, the Objective-C mirror generator emits wrapped types only for properties, method result types and method parameter types that are *not* annotated with either `nonull` or `_Nonnull` in the original Objective-C code.

Suppose the example from the introduction to this section was enhanced with nullability annotations as follows:

```
@interface MyContainer: NSObject
   .  .  .
- (void)addItem:(nonnull MyItem *)item withUuid:(nonnull NSString *)uuid;
- (nullable MyItem *)itemWithUuid:(nonnull NSString *)uuid;
- (nullable NSString *)uuidForItem:(nonnull MyItem *)item;
@property (copy, nonnull) NSArray<MyItem *> *allItems;
@end
```

The mirror generator would then bypass `Option<T>` wrapping for the `nonnull` entities (`@ForeignName` annotations omitted for brevity):

```
@ObjCMirror
open class MyContainer <: NSObject {
       .  .  .
    public open fund addItemWithUuid(item: MyItem, uuid: NSString): Unit
    public open func itemWithUuid(uuid: NSString): ?MyItem
    public open func uuidForItem:(item: MyItem): ?NSString
    public open prop allItems: NSArray
}
```

NOTE: It is currently not possible to mirror `null_resettable` properties into Cangjie properties while propagating the semantics of that annotation, hence the latter is treated exactly as `nullable`.

If your Objective-C code that you want to access from Cangjie is not taking advantage of the nullability annotations, you may want to add at least the `nonnull` annotations as appropriate before you begin using the Cangjie SDK interoperability features described here. That may considerably reduce the use of `Option<T>` wrapping in the respective mirror types, making the code of interop classes cleaner and easier to read.

Objective-C Mirror Generator Reference

Prerequisites

Make sure to run the `envsetup.sh` script from the Cangjie SDK before using the mirror generator.

You need to know the pathnames of the header files of the frameworks and libraries in which all dependencies of the classes you are going to mirror are defined. This includes the locations of the iOS standard library headers and generally all directories in which the Objective-C compiler looks up header files when it builds your project.

Command-line Syntax

`ObjCInteropGen [-v] [--mode=normal` *`config-file`*`]`

`-v`

Produce verbose output.

`--mode=normal`

Enforces the normal operation mode. Other modes are only used for the development and testing of the mirror generator itself. The use of `--mode=normal` is *mandatory* in the current version if *`config-file`* is specified, but will be optional in the future.

`config-file`

The pathname of the configuration file.

Configuration File Syntax

An Objective-C mirror generator configuration file is a plain text file in the TOML syntax. It specifies:

- Output directories
- Names of the source Objective-C headers (`.h`-files)
- Names of the output Cangjie packages and the distribution of mirror types across them
- Mappings for types that you want to be handled in a special way

String values that are interpreted as regular expressions must follow the ECMAScript regular expression syntax.

Output Directories Roots

Each entry in the `[output-roots]` table of tables defines a symbolic name for the pathname of a directory in the local file system. The mirror generator will use that pathname as the common root of one or more package-specific output directories, set using the `output-root` property of the respective `[[packages]]` array elements.

Example:

```
[output-roots.lib]
path = "./lib/src"
```

```
[output-roots.app]
path = "./main/src"

[[packages]]
package-name = "com.vendor1.lib1"
output-root = "lib"  # Output to "./lib/src/com/vendor1/lib1"
filters = ...

[[packages]]
package-name = "com.vendor2.lib2"
output-root = "lib"  # Output to "./lib/src/com/vendor2/lib2"
filters = ...

[[packages]]
package-name = "com.mycompany.app"
output-root = "app"  # Output to "./main/src/com/mycompany/app"
filters = ...
```

Source Files

Each entry in the `[sources]` table of tables specifies a set of individual header files that the mirror generator must take as input.

Properties:

`paths` (mandatory)

An array of strings that contain pathnames of individual header files that the mirror generator must read.

`arguments` (optional)

Array of strings containing Clang options that the mirror generator will pass over when processing the source files listed in `paths`.

Example:

```
[sources.all]
paths = ["original-objc/M.h"]
```

Additional Clang Arguments

Each entry in the `[sources-mixins]` table of tables specifies a regular expression matching one or more keys of the `[sources]` table and additional arguments that must be passed to Clang when processing the header files from its respective entries.

Properties:

`sources` (mandatory)

A string containing a regular expression.

`arguments-prepend`
`arguments-append` (optional)

Arrays of strings that the mirror generator will pass over to Clang as options when processing the source files listed in any `[sources]` table entry which key matches the regular expression specified in the `sources` property.

The options listed in `arguments-prepend` and `arguments-append` will be passed respectively before and after the options specified in the `arguments` property of each matching `[sources]` entry, if any.

Example:

```
[sources.UIWidgets]
paths = ["objc/UIWidgets.h"]
arguments = [ "-I", "/usr/local/include/share/Widgets" ]

[sources.UIPanels]
paths = ["objc/UIPanels.h"]
arguments = [ "-I", "/usr/local/include/share/Panels" ]

[sources-mixins.UI]
# Add these Clang arguments for both UIWidgets and UIPanels
sources = ["UI.+"]
arguments-append = [
    "-I", "/usr/local/include/Frameworks/AcmeUI"
]
```

Packages

Each entry in the `[[packages]]` array specifies a target Cangjie package name, a set of name filters that define which Objective-C entities will be mirrored to that package, and, optionally, the output directory specific for that package.

Properties:

`package-name` (mandatory)

A string containing the target Cangjie package name.

`output-path` (optional)

A string containing the *exact* pathname of the directory into which the mirror generator shall place the output files for the given package. If any directories in that pathname do not exist, the mirror generator will attempt to create them.

`output-root` (optional)

A string containing a key from the `output-roots` table The name of the target Cangjie package will be appended to the value of the respective `output-roots`

table entry, dots `.` replaced with the file separator `/`, and the resulting pathname will be used as if it was the value of `output-path`.

If `output-root` is not set, and if there is only one element in `[output-roots]`, the pathname from that entry is used. Otherwise, an error is shown.

Example:

```
[output-roots.main]
path="./cj-mirrors"

[[packages]]
package-name = "objc.foundation"
output-root = "main"
```

The output files will be placed in `./cj-mirrors/objc/foundation`.

`filters` (mandatory)

A table defining a set of name filters that select only those Objective-C entity declarations from the source files that must be mirrored to the given Cangjie package. See Name Filters for details.

Example:

```
# The Foundation framework
[[packages]]
package-name = "objc.foundation"
filters = { include = "NS.+" }
```

Name Filters Each name filter is, in turn, a TOML table that contains:

- Exactly one of the properties `include`, `exclude`, `union`, `intersect` and `not`, and
- Optionally, the `filter` property *and/or* the `filter-not` property.

Descriptions of the properties follow.

`include`

The value can be a regular expression or an array thereof. If the value is a single regex, only names that match it pass the filter. If the value is an array, a name only needs to match any single regex from that array to pass.

Examples:

```
# Only include entities that names of which start with "NS":
filters = { include = "NS.+" }

# Only include entities with names that either start with "Foo"
# or end with "Bar":
filters = { include = ["Foo.*", ".*Bar"] }
```

`exclude`

The opposite of `include` (see above). A name that would pass an `include` filter with the given value does not pass an `exclude` filter with the same value and vice versa.

Example:

```
# Include everything but entities the names of which start with "INTERNAL_":
filters = { exclude = "INTERNAL_.+" }
```

`union`

An operator combining two or more filters. Its value must be an array of filters. A name that passes any single one of those filters passes the entire `union` filter.

Example:

```
# An equivalent of the second `include` example above:
filters = { union = [ { include = "Foo.*" }, { include = ".*Bar" } ] }
```

`intersect`

An operator combining two or more filters. The value must be an array of filters. A name must pass all of them to pass the the entire `intersect` filter.

Example:

```
// Adding a negative filter:
filters = { intersect = [ { include = "NS.+" },
                          { exclude = "NSAccidentalClash" } ] }
```

`not`

An operator that reverses the meaning of a filter. The value must be a single filter:

Example:

```
// Another way to add a negative filter:
filters = { intersect = [ { include = "NS.+" },
                          { not = { include = "NSAccidentalClash" } } ] }
```

`filter`
`filter-not` (optional)

These properties must be mixed with other properties, i.e. they cannot be the only properties of a `filters` table. Their values can be regular expressions or arrays thereof, just as those of the `include` and `exclude` properties.

`filter` reduces the set that passed the main filter to include only names that match the given single regular expression or any regex from the array.

`filter-not` reduces the set that passed the main filter to include only names that do not match the given single regular expression or any regex from the array.

`filter` and `filter-not` are actually shorthands for `intersect` operations with an `include` and `exclude` filter respectively. They can be specified together.

Examples:

```
# Without filter-not:
filters = { intersect = [ { include = "NS.+" },
                          { exclude = "NSAccidentalClash" } ] }
# With filter-not:
filters = { include = "NS.+", filter-not = "NSAccidentalClash" }

# 'filter' and 'filter-not' can be used together:
filters = { include    = ".*Fizz.+",
            filter     = ".+Buzz.*",
            filter-not = ".*FizzBuzz.*" }
```

Type Substitutions

`[[mappings]]` is an array of tables, each adding to the list of substitutions of one Objective-C type for another. All mentions of almost any type (except for C primitive types such as `int`) can be replaced with mentions of another type.

Example:

```
# Replace the type id, the root of the Objective-C type hierarchy,
# with NSObjectProtocol everywhere.
[[mappings]]
id = "NSObjectProtocol"
```

Configuration File Import

`imports` is an array of strings each containing the pathname of another configuration file, the settings from which will be added to the current configuration. The entries of `packages` and `mappings` arrays found in the imported file are appended to those present in the importing file.

Nested imports are supported, circular import is detected and results in an error.

Example:

```
import = "../common.toml"
```